

GraSP: Secure Collaborative Graph Processing Made Scalable

Siddharth Kapoor
University of Utah
siddharth00kapoor@gmail.com

Nishat Koti
Aztec Labs
nishat@aztec-labs.com

Varsha Bhat Kukkala
IIT Tirupati
varshabhat@iittp.ac.in

Arpita Patra
Indian Institute of Science
arpita@iisc.ac.in

Bhavish Raj Gopal
Indian Institute of Science
bhavishraj@iisc.ac.in

Abstract

Secure graph processing enables computing on graphs while hiding the graph topology as well as the associated node/edge data. This facilitates collaborative analysis among multiple data owners, who may only hold a private, partial view of the global graph. Several works address this problem using the technique of secure multiparty computation (MPC) in the presence of 2 or 3 parties. However, when moving to the multiparty setting, as required for collaborative analysis among multiple data owners, the existing solutions are no longer scalable. This remains true with respect to the state-of-the-art framework of Koti et al. (CCS 2024) as well. Specifically, the state-of-the-art incurs a round complexity linear in the number of parties or data owners. This is due to its reliance on secure shuffle protocol, constituting a bottleneck in the multiparty setting. Additionally, it has a prohibitively expensive initialisation phase due to its reliance on secure sort, with a round complexity dependent on both the graph size and the number of parties.

To address the above limitations in the multiparty setting, we first propose a novel Decompose-Compute-Combine (DCC) paradigm for efficient computation on graph-structured data. We then design GraSP, a generic framework for secure graph processing that leverages the DCC paradigm to improve efficiency and scalability for the multiparty setting. Elaboratively, GraSP is designed to (1) eliminate the need of shuffle and instead rely on a weaker primitive known as Permute+Share, and (2) have a lightweight initialisation, which eliminates the need for secure sorting. This allows GraSP to achieve a round complexity that is completely independent of the graph size and number of parties, making it highly scalable. Finally, we implement and benchmark the performance of GraSP for the application of PageRank computation and showcase its efficiency and scalability. Concretely, we witness improvements of up to 78 $\times$ in runtime compared to the state-of-the-art. Further, we observe that GraSP takes under a minute to perform 10 iterations of PageRank computation on a graph of size 10^6 , distributed among 25 data owners, making it highly practical for secure graph processing in the multiparty setting.

Keywords

Secure graph processing, secure computation

1 Introduction

Graphs are fundamental for representing and analysing complex relationships in diverse real-world applications. In many applications these graphs contain sensitive user information and are

usually distributed across multiple data owners. In such scenarios, processing the global graph while hiding its topology and the associated sensitive data is a challenge. For example, consider a financial transaction graph where nodes (or vertices) represent bank accounts, and edges represent transactions. Each bank (data owner) is privy to only part of the transaction graph, related to their own registered accounts (vertices) and the corresponding transactions (edges). Nonetheless, banks would be interested in processing the global transaction graph for various reasons, such as identifying fraudulent transactions, credit risk analysis, etc. However, due to privacy constraints, regulations, and commercial interests, banks cannot simply share their respective information with each other. Thus, there is a need for designing techniques that allow data owners to collaboratively process sensitive and distributed graph data while ensuring individual subgraph data and topologies are not revealed. We refer to this as *secure collaborative graph processing*. With this as the motivation, the current work aims to design an efficient framework that allows multiple data owners with private subgraphs to collaboratively process the global graph such that each data owner learns no additional information other than the final result. We rely on the technique of secure multiparty computation (MPC) to design a solution for the same. This technique allows a set of n parties to compute a function on their private inputs such that even if some t parties, with $t < n$, collude with one another, they learn no information beyond the output. In the context of secure collaborative graph processing via MPC, data owners can enact the role of parties, with their local views of the graph as private inputs, and the graph computation corresponds to the function. To contextualise our contribution, we first outline the existing approaches in the literature.

To design efficient solutions, existing generic graph computation frameworks [1, 11, 15] (capable of realising a large class of graph algorithms) leverage the sparsity of real-world graphs. They operate on a list-based graph representation to securely realise any message-passing graph algorithm. The latter are stateful iterative algorithms where each node/edge maintains state information. Each iteration involves the following: (i) propagating information from vertices onto their incident edges; (ii) receiving and aggregating data from incident edges onto vertices; (iii) using the aggregated data to update the current state of each vertex. These operations performed in each iteration of the message-passing algorithm are referred to as Scatter, Gather and Apply respectively. [11] forms the state-of-the-art MPC framework for securely realising message-passing algorithms. Leveraging the fact that many widely used graph algorithms rely on linear aggregation functions (in step (ii)),

[11] achieves round complexity independent of the graph size, unlike prior works, making it highly efficient. Even for non-linear aggregation, [11] outperforms prior MPC solutions [1, 15].

Note that [11] is designed primarily for the outsourced computation setting. In this setting, data owners do not directly participate in the MPC computation. Instead, they delegate these computations to a small set of servers hired on a pay-per-use basis. Privacy is ensured as long as at least one server remains honest and does not collude with other servers or data owners. However, this model may not be suitable for all scenarios, as it requires data owners to relinquish direct control over their data and trust that the non-collusion assumption holds. Further, the data owners may be distributed across regions with stringent privacy regulations that restrict data movement beyond regional boundaries. In such cases, it may be infeasible to identify and agree on a small set of servers to outsource the computation. In contrast, a non-outsourced multiparty computation setting addresses these issues. Here, data owners themselves enact the role of MPC parties, retaining complete control over their data. Specifically, the privacy of an honest data owner is guaranteed, even if all other data owners collude. Thus, a non-outsourced setting may be more desirable in scenarios involving multiple data owners with strict privacy requirements.

Alternatively, [19] proposes a solution for a non-outsourced setting. They specifically consider secure training and inference of graph neural networks (GNNs) in a multi-party setting. While their framework can, in theory, support any message-passing graph algorithm, it assumes a highly restrictive security model in which no two parties participating in the protocol are allowed to collude. By adopting this weaker security model, [19] aims to optimise efficiency. [20] builds on top of [19] and designs a solution that has communication complexity independent of the number of parties. However, it still relies on the restrictive threat model, which we believe is impractical. To account for the aforementioned scenarios, [11] can be instantiated in the non-outsourced setting by using a generic MPC protocol that supports an arbitrary number of parties. However, unlike the outsourced setting, which benefits from customised small-party computation protocols, generic MPC protocols incur higher overheads in terms of both rounds and communication, thereby affecting scalability. Specifically, in the non-outsourced multiparty scenario, [11] suffers from two main limitations—(i) it requires an expensive initialisation phase that incurs a round complexity that is linearly dependent on both the graph size and the number of parties due to its reliance on secure sort protocol; additionally, communication complexity is log-linear in graph size, (ii) each iteration of message-passing incurs a round complexity that is linearly dependent on the number of parties owing to the need for a secure shuffle protocol. A higher round and communication complexity is a critical bottleneck, as it directly impacts the runtime of the protocol in the multiparty setting. For instance, parties could be geographically distributed over a WAN. Here, the high round-trip time (RTT) and low bandwidth, coupled with the higher round and communication complexity of the MPC protocols, can result in impractically long runtimes for securely performing graph computations.

Thus, keeping key factors of efficiency, scalability and strong security guarantees in mind, the current work aims to design secure

graph processing in the presence of multiple data owners while overcoming the drawbacks of prior works.

1.1 Contributions

We design GraSP(Graph Secure Processing), an efficient end-to-end framework for secure graph processing in the distributed setting to realise any graph algorithm expressed as message-passing graph algorithm. We first design a novel data-oblivious paradigm to efficiently realise message-passing graph algorithms when there are multiple data owners. We then translate this into a secure solution for collaborative graph processing via MPC. Note that our MPC protocols, which make up GraSP, are designed to introduce efficiency improvements by incurring low round complexity (independent of the number of data owners or graph size), resulting in the desired scalability. Since efficiency takes centre stage, GraSP leverages the preprocessing paradigm in MPC, where input-independent MPC computations are offloaded to a preprocessing phase so that input-dependent computations performed in the online phase can be carried out extremely fast. The distinction between the two phases is crucial in practice, as the online phase determines the protocol's response time, while the preprocessing phase can be performed well in advance without impacting the response time. Given the significance of achieving a fast response time, unless otherwise stated, the complexities stated subsequently refer to the online complexities. Our contributions are detailed below.

Novel design paradigm of GraSP: We design a new and efficient Decompose-Compute-Combine (DCC) paradigm for securely realising any message-passing graph algorithm on the global graph $\mathcal{G} = (V, E)$ that is distributed. The paradigm involves performing the following: (i) decomposing $\mathcal{G}$ into n subgraphs $\mathcal{G}_i$ for $i = 1$ to n , where $\mathcal{G}_i$'s topology is known to a distinct party P_i , (ii) carrying out multiple iterations of the message-passing computations on the individual subgraphs $\mathcal{G}_i$ in parallel, instead of doing the same on the global graph $\mathcal{G}$, (iii) combining the results on these subgraphs to obtain results on the global graph $\mathcal{G}$. GraSP leverages the fact that each party (data owner) has knowledge of its own vertices, edges incident on them, and the neighbouring vertices, all of which comprise the subgraph $\mathcal{G}_i$ to be operated on. The DCC paradigm is designed in such a way that, when translated into the MPC setting, it yields significant efficiency benefits. In particular, steps (i)-(iii) aid in achieving a round complexity *independent of the number of parties* n , allowing GraSP to scale efficiently for multiparty settings.

Lightweight initialisation phase: When designing the secure solution, computations that remain consistent across the iterations of the message-passing graph algorithm are moved into a one-time initialisation phase. This initialisation phase is prohibitively expensive in the prior works, including the state-of-the-art work of [11]. This is due to the need for securely sorting the list-based representation of the graph, which is known to be prohibitively expensive in the multiparty setting. In contrast, our solution has a lightweight initialisation that completely eliminates the need for secure sorting. In fact, the initialisation in GraSP only entails secret sharing its data among the participating parties. Consequently, our initialisation phase achieves constant round complexity, independent of the graph size and number of parties. Further, it achieves an improved communication complexity of $O(nN)$, linear in both the number of

parties n and the graph size N , where N denotes the total number of vertices and edges in $\mathcal{G}$. In contrast, the state-of-the-art, [11], incurs a round complexity dependent on the number of parties and the graph size, i.e., $O(n + \log(N))$. Furthermore, it has a communication complexity that is log-linear in the graph size, i.e., $O(n \log(N))$. A detailed comparison appears in Table 1.

Improved protocol for message-passing: A message-passing algorithm iteratively updates the state of vertices in $\mathcal{G}$ for realising the graph computation. In the secure realisation, each iteration of message-passing computation requires securely reordering the list representing the graph to perform the Scatter, Gather, and Apply computations. In prior works [11, 15], this reordering incurs a round complexity that is dependent on the number of parties, n . This dependence on n arises from the reliance on a secure shuffle protocol, which allows secret shares of a list (in this case, the list representation of $\mathcal{G}$) to be reordered based on a random permutation not known to any party. This protocol incurs $O(n)$ rounds in the multiparty setting [8]. On the contrary, in our approach, the message-passing computations occur over subgraphs whose topology is known to one of the parties. Leveraging this information, we modify and enhance each iteration, thereby avoiding the need for an expensive shuffle protocol. Instead, for our designed solution, it suffices to rely on a more efficient Permute + Share protocol that can be realised in $O(1)$ rounds, even in the multiparty setting.

To give a more detailed characterisation of the round complexity, note that both [11] and GraSP realise each iteration of message-passing computations through the primitive operations of Propagate, ApplyE (together referred to as Scatter), Gather and ApplyV. Informally, Propagate transfers data from the source vertex to its outgoing edges. ApplyE, denoted AE , updates the data on the edges. Gather aggregates the data received on the incoming edges of a vertex, and ApplyV denoted AV , updates the vertex's state based on the aggregated data. In [11], this incurs a round complexity of $O(n + r_{AE} + r_{AV})$, where r_x denotes the round complexity of securely computing x . On the other hand, GraSP incurs round complexity independent of the number of parties, i.e., $O(r_{AE} + r_{AV})$ while maintaining the same communication as that of [11]. A comparison of GraSP with [11] is provided in Table 1.

Phase	Ref	Rounds	Communication
Initialisation	Graphiti [11]	$O(n + \log N)$	$O(n \cdot N \cdot \log N)$
	GraSP	$O(1)$	$O(n \cdot N)$
Message-passing	Graphiti [11]	$O(n + r_{AE} + r_{AV})$	$O(n \cdot N + (c_{AE} + c_{AV}) \cdot N)$
	GraSP	$O(r_{AE} + r_{AV})$	$O(n \cdot N + (c_{AE} + c_{AV}) \cdot N)$

Given graph $\mathcal{G} = (V, E)$, $|V|$ denotes number of vertices, $|E|$ denotes number of edges and $N = |V| + |E|$. r_x and c_x denote the round and communication complexity of computing x securely via MPC.

Table 1: Comparison of the initialisation and one iteration of message-passing graph algorithm.

Amortised Permute+Share: Naively using Permute + Share primitive [4] in our framework results in round complexity that is independent of n , but it introduces a quadratic communication overhead, i.e., $O(n^2 \cdot N)$. This is because our design requires permuting the same input list according to n different permutations, each known to a distinct party. A naive approach requires n parallel invocations of Permute + Share, with each invocation requiring $O(n \cdot N)$ communication. Instead, by leveraging the fact that the input list remains the same across all n invocations, we design

Amortised Permute + Share that significantly reduces the online complexity. This protocol has same online communication as that of a single invocation of Permute + Share while maintaining constant round complexity. Elaborately, the Amortised Permute + Share protocol takes as input a secret-shared list and outputs n secret-shared lists, where each list is permuted according to a permutation known to a distinct party. Note that the Amortised Permute + Share protocol can be of independent interest, as it enables multiple permutations to be performed on a secret-shared list efficiently.

Benchmarks: We implement and benchmark GraSP for PageRank computation to demonstrate its practicality and improvements over the state-of-the-art [11]. Specifically, we compare the costs of GraSP with that of [11] while varying graph size ($N = |V| + |E|$) and number of parties. We observe improvements of 78 $\times$ in runtime and 20 $\times$ in communication for a graph of size 10^5 distributed among 15 parties. Moreover, we note that GraSP is highly scalable and takes under a minute to complete PageRank computation on a graph of size 10^6 distributed among 25 parties, while in the considered experimental setup, [11] could not support more than 15 parties.

2 Preliminaries

System and Threat model: We consider a directed graph $\mathcal{G} = (V, E)$ that is distributed across n parties (data owners) in $\mathcal{P}$. Each party $P_i \in \mathcal{P}$ owns a subset of vertices or nodes, denoted as V_i , such that $V = \bigcup_{i=1}^n V_i$. Each vertex $v \in V$ is associated with a public label, $v.id$, where the label as well as the information about which party owns this vertex is known to all parties. Thus, V is known to all the parties. Each vertex v may also be associated with a data component denoted as $v.data$, and is private to the party that owns v . Each directed edge $e \in E$ consists of $(e.src, e.dst)$, representing the source and destination label, respectively. An edge e may also have a data component denoted as $e.data$. Information about existence of an edge e where both $e.src, e.dst \in V_i$, is known only to P_i . Its associated data $e.data$ is also private only to P_i . For edges where $e.src \in V_i$ and $e.dst \in V_j$, the existence of e and its associated data $e.data$ is known to both P_i and P_j . We let E_i represent the set of all edges $e \in E$ where $e.dst \in V_i$, and have $E = \bigcup_{i=1}^n E_i$.

The parties are interested in performing computations on the global graph $\mathcal{G}$, while ensuring their graph data remains private from other parties. We assume that the parties are each connected via pairwise private and authenticated channels in a synchronous network. In this work, we consider the parties to be honest-but-curious or semihonest, where they follow the protocol honestly while trying to gain more information. Furthermore, we also consider that any subset of up to $n - 1$ parties may collude to learn information about the inputs of other parties. We model this using a static, semi-honest, probabilistic, polynomial-time adversary $\mathcal{A}$ capable of corrupting up to $n - 1$ parties in $\mathcal{P}$. We prove security using the standard real-world/ideal-world simulation paradigm. Our framework operates over a ring algebraic structure, with $\mathbb{Z}_{2^\ell}$ denoting the ring of ℓ -bit elements.

Secret-Sharing and Multiparty Computation: Firstly, we assume that the data owners map the graph data to elements on a ring algebraic structure, with $\mathbb{Z}_{2^\ell}$ denoting the ring of ℓ -bit integers. We say a value $x \in \mathbb{Z}_{2^\ell}$ is $\langle \cdot \rangle$ -shared (or additively shared) over $\mathbb{Z}_{2^\ell}$ if each party $P_i \in \mathcal{P}$ holds a random share $\langle x \rangle_i \in \mathbb{Z}_{2^\ell}$ such

that $x = \langle x \rangle_1 + \dots + \langle x \rangle_n$. This secret sharing scheme is linear since, given shares of $x, y \in \mathbb{Z}_{2^\ell}$ and public constants $c_1, c_2 \in \mathbb{Z}_{2^\ell}$, parties can non-interactively generate shares of $c_1x + c_2y$. However, non-linear operations such as multiplication or comparison require communication among the parties. We rely on a one-time key setup [2, 5, 12, 14, 16] that establishes common random keys for a pseudo-random function (PRF) among the parties. Using these keys, subsets of parties can non-interactively sample a common random ℓ -bit string $v \in \mathbb{Z}_{2^\ell}$. This enables a party to non-interactively generate $\langle \cdot \rangle$ -shares of a value x as described in §A. However, the reconstruction of a $\langle \cdot \rangle$ -shared value towards P_i requires interaction where every party communicates its share to P_i .

To enable secure computation over shares, we rely on a secret-sharing-based MPC protocol [6] which allows the parties to perform computation on secret-shared inputs to generate secret-shared outputs. Specifically in this work, we rely on the following primitive operations, which are abstracted via ideal functionalities. (i) Multiplication: This primitive takes as input two $\langle \cdot \rangle$ -shared values $x, y \in \mathbb{Z}_{2^\ell}$ and outputs the $\langle \cdot \rangle$ -shares of $z = x \cdot y$. We let $\mathcal{F}_{\text{mult}}$ denote the ideal functionality for the same. (ii) Permute and share: This primitive takes as input a $\langle \cdot \rangle$ -shared vector $T \in \mathbb{Z}_{2^\ell}^N$ of size N , where each element in T , i.e., $T[i], \forall i \in \{1, \dots, N\}$, is $\langle \cdot \rangle$ -shared and a permutation π from one party. It outputs $\langle \cdot \rangle$ -shares of a vector $T_O \in \mathbb{Z}_{2^\ell}^N$ such that $T_O = \pi(T)$. We let $\mathcal{F}_{\text{Permute+Share}}$ denote the ideal functionality for it. (iii) The underlying MPC: Note that our protocols may require various other MPC primitives such as comparison, equality etc. depending on the considered graph algorithm. We abstract these out via the $\mathcal{F}_{\text{MPC}}$ functionality, which takes secret-shared inputs from the parties and returns a secret-shared output of the function computed on the inputs. We instantiate $\mathcal{F}_{\text{MPC}}$ via the generic MPC protocol of [6] in the semihonest setting.

Secure Shuffle and Permute+Share. Let $T \in \mathbb{Z}_{2^\ell}^N$ be a list of N entries, where each entry is from $\mathbb{Z}_{2^\ell}$. The list T is additively secret-shared *entry-wise* among all parties in $\mathcal{P}$. Let $\langle T \rangle_i \in \mathbb{Z}_{2^\ell}^N$ denote the share of party $P_i \in \mathcal{P}$. Secure shuffle's aim is to take $\langle \cdot \rangle$ -shares of T as input and generate $\langle \cdot \rangle$ -shares of a shuffled list $T_O \in \mathbb{Z}_{2^\ell}^N$, where $T_O = \pi(T)$, i.e., T shuffled using a random secret permutation π (not known to any party) where $T_O[i] = T[\pi(i)]$.

Secure shuffle can be constructed using a weaker building block called Permute + Share, as described in [4, 8]. The Permute + Share protocol allows shuffling a secret-shared list using a permutation known to one party. Specifically, the Permute + Share protocol takes as input a secret-shared list T and a permutation π_i known to a designated party P_i , and outputs $\langle T_O \rangle$ such that $T_O = \pi_i(T)$. Secure shuffle can then be realised by performing n sequential invocations of Permute + Share, where each party contributes a random permutation ensuring that the final permutation applied is a composition of all permutations that is not known to any party.

The Permute + Share protocol described in [4] consists of two phases: preprocessing and online phase. During the preprocessing phase, the parties generate additive secret shares of a random list R (not known to any party) and its permuted counterpart $\pi_i(R)$. These shares can be generated using pairwise invocation (between P_i and every other party) of a 2-party Share Translation protocol, as detailed in [4, 8]. Each invocation of Share Translation protocol incurs a communication of $O(N)$, making the total preprocessing

communication cost for n invocations $O(nN)$. Given, $\langle R \rangle, \langle \pi_i(R) \rangle$ generated in the preprocessing, the parties reconstruct $T+R$ towards the designated party P_i . All parties except P_i then set their output shares of T_O as their respective shares of $\pi_i(R)$ i.e. party P_j for $j \neq i$ sets $\langle T_O \rangle_j = -\langle \pi_i(R) \rangle_j$. Party P_i computes its output share as $\pi_i(T+R) - \langle \pi_i(R) \rangle_i$. This ensures that the output list T_O satisfies $T_O = \pi_i(T)$. This requires one round and $O(nN)$ communication.

Naively realising secure shuffle via Permute + Share involves n sequential invocations of the Permute + Share protocol, with each party contributing one random permutation. This approach results in $O(n)$ rounds of communication and $O(n^2N)$ total communication cost during the online phase. The preprocessing cost similarly involves $O(n^2)$ invocations of Share Translation, resulting in a communication cost of $O(n^2N)$. The work of [8] improves this naive realisation by optimising the online phase communication to $O(nN)$ while maintaining the same number of rounds and similar preprocessing costs. We refer interested readers to [8] for further details of the optimised protocol.

Secure computation of graph algorithms: Several graph algorithms such as BFS [1, 11] DFS, PageRank (see C), centrality measures [10], clustering [10], GCN [9] etc., can be naturally represented as message-passing algorithms. There are several works that consider secure realisation of message-passing graph algorithms [1, 11, 15, 19, 20]. We next describe the details of the list representation of the graph used in these works, followed by data-oblivious algorithms for the primitive operations of scatter, gather and apply. *Data-augmented list representation.* In these works, the input graph $\mathcal{G} = (V, E)$ is represented as a data-augmented graph (DAG) list or DAG-list G which includes an entry for each vertex and edge in $\mathcal{G}$. Each entry consists of a tuple (src, dst, isV, data) of four components: source (src), destination (dst), is_Verex (isV) to distinguish entries of a vertex from an edge, and data to store various data items associated with the vertex/edge. Specifically, a node u is represented as $(u, u, 1, \text{data})$, and a directed edge $e(u, v)$ is represented as $(u, v, 0, \text{data})$. The data field can contain multiple sub-components, some specific to vertices and others specific to edges. To ensure the topology of the graph is hidden, every entry in the DAG-list includes all sub-components, with irrelevant ones set to 0 or dummy values. Undirected graphs can also be represented as DAG-list by accounting for every edge twice in both directions.

Data oblivious computations on DAG-list. A key requirement for secure realisation of a graph algorithm is that the execution of the graph algorithm must reveal *no information* about the underlying topology beyond what is explicitly permitted by the security model. A data-oblivious algorithm guarantees that the sequence of memory accesses, the pattern of control flow, and the order in which entries of DAG-list are touched depend *only* on public parameters (e.g., $|V|, |E|$) and not on the actual structure of the graph. To perform data-oblivious message passing computations across iterations as required for the underlying graph algorithm, these frameworks rely on the following set of primitives¹: (i) Propagate: enables vertices to propagate data along their outgoing edges (ii) ApplyE: enables update of the edge data based on a user-defined function f_{AE} , (iii) Gather: enables vertices to aggregate data from its incoming edges,

¹Propagate and ApplyE are together referred to as Scatter

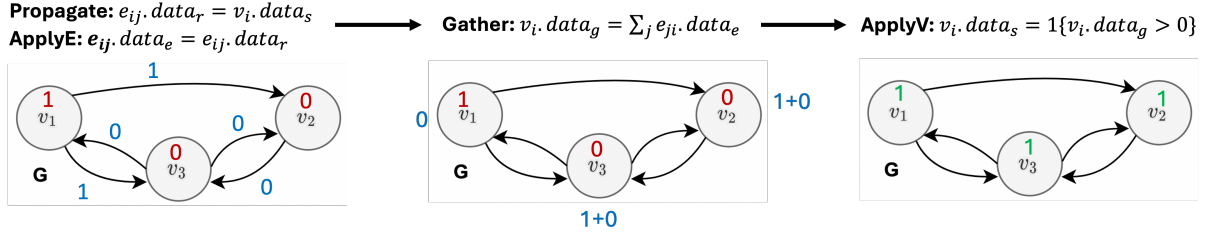


Figure 1: Example of one iteration of message passing computation to identify one-hop neighbours of the source vertex v_1 . k -iterations reveal all vertices within k -hop distance.

and (iv) ApplyV: enables update of all vertices based on a user-defined function f_{AV} ². To efficiently realise these primitives in a data-oblivious manner, the state-of-the-art framework of [11] relies on three orderings of the DAG-list. In the vertex order, all vertex entries in the list appear first, followed by all edge entries. In the source order, every vertex entry in the list appears immediately before entries of all its outgoing edges. Similarly, in the destination order, entries of all the incoming edges of a vertex appear immediately before the entry of the corresponding vertex. Using these different orderings, each iteration of message-passing computation is achieved as follows. It begins by invoking Propagate that entails performing computation on the vertex order of DAG-list, followed by computations in the source-ordered DAG-list. Hence, this requires reordering the DAG-list from vertex to source order. This is followed by ApplyE, which ensures data on each edge is updated under the function f_{AE} . Since this operation is local to each entry of the graph, this operation can be computed in parallel at all entries of the DAG-list and also does not require any reordering. This is followed by invoking Gather, which entails performing computations over destination order, followed by computations in the vertex-ordered DAG-list. Thus, Gather requires reordering the DAG-list from source order to destination order as well as a reordering back to vertex order. Finally, an invocation to ApplyV ensures data on each node is updated under the function f_{AV} . Similar to ApplyE, this operation is local to each entry of the graph. Taken together, these steps yield a fully data-oblivious realisation of one iteration of message passing. Algorithm 1 gives the formal details of the data-oblivious algorithm proposed by [11]. A pictorial illustration of one iteration of message-passing computation appears in Fig. 1. We refer an interested reader to §A.2 and [11] for the intuition and correctness of the algorithm.

Secure realisation of message-passing computations. Secure realisation of Algorithm 1 using MPC on a secret-shared DAG-list G enables the secure realisation of one iteration of message-passing computations and subsequently the graph algorithm. In the secure realisation of Propagate and Gather, the computations performed on the DAG-list consist of linear operations and hence do not require interactions between parties when realised via MPC, i.e., given secret-shares of the DAG-list, the computations involved in steps 2, 5 and 10, 13 can be performed non-interactively by leveraging the linearity of secret-sharing. Thus, the only interactive operation involved is the reordering of the DAG-list between different orderings (in steps 3, 8, 11, and 14) and updating the edge and vertex data (in steps 7 and 16). The computations in ApplyE (step 7) and ApplyV

Algorithm 1: One iteration of message-passing

Input: DAG-list G in vertex order.

Each entry contains data components: $data_s, data_g, data_v, data_e, data_r$. Here, $data_s$ denotes data propagated from vertices to edges, $data_g$ aggregates incoming messages, and $data_v, data_e$ denote vertex and edge data. $data_r$ is (temporary) to hold intermediate computations. Aggregation operation is addition.

Output: DAG-list G in vertex order with updated data

```

/* - Propagate operation - */
1 for  $i \leftarrow |V|$  to 2 do
2    $G[i].data_r \leftarrow G[i].data_s - G[i-1].data_s$ ;
3 Reorder  $G$  to the source order;
4 for  $i \leftarrow N$  to 1 do
5    $G[i].data_r \leftarrow \sum_{j=1}^i G[i].data_r - G[i].data_s$ ;
/* - ApplyE operation - */
6 for  $i \leftarrow N$  to 1 do
7    $G[i].data_e \leftarrow f_{AE}(G[i].data_e, G[i].data_r) \cdot (1 - isV)$ ;
/* - Gather operation - */
8 Reorder  $G$  to the destination order;
9 for  $i \leftarrow 1$  to  $N$  do
10   $G[i].data_g \leftarrow \sum_{j=1}^i G[i].data_r$ ;
11 Reorder  $G$  to the vertex order;
12 for  $i \leftarrow N$  to 2 do
13   $G[i].data_g \leftarrow G[i].data_g - G[i-1].data_g$ ;
14 Reorder  $G$  to the vertex order;
/* - ApplyV operation - */
15 for  $i \leftarrow 1$  to  $|V|$  do
16   $G[i].data_v \leftarrow f_{AV}(G[i].data_v)$ ;

```

(step 16) can be realised in parallel for each entry of the DAG-list G and are dependent on the functions f_{AE} and f_{AV} , respectively. A straightforward approach to reorder the DAG-list is to use a secure sorting protocol [15]. However, subsequent work has proposed optimisations to avoid repeated sorting. The key observation in [1] is that the sequence of required reordering (vertex $\rightarrow$ source $\rightarrow$ destination $\rightarrow$ vertex) remains fixed across all message-passing iterations. Therefore, these permutations can be securely precomputed once during an initialisation phase using a secure sort. Thereafter, reordering can be performed by applying the permutation using secure shuffle (see A.2 for further details).

Notations. For clarity of presentation, we summarise the key symbols and terminology used throughout the paper in Table 2.

²The framework allows adapting the primitives for edges in any direction.

3 DCC approach for message passing

As discussed earlier, the existing works do not scale well in the multiparty setting due to *expensive initialisation phase* and *unscalable message-passing*. The design of existing works overlooks the fact that each data owner naturally possesses local information about its own vertices and adjacent edges, which could be exploited to optimise computation and communication. In contrast, our Decompose-Compute-Combine paradigm offers a novel approach for message-passing computations, specifically designed to leverage local information while maintaining correctness and privacy. This, in turn, results in a secure solution for graph processing that is both lightweight and scalable for a large number of parties.

3.1 Overview of DCC Paradigm

Consider one iteration of message-passing computation on a graph $\mathcal{G}$ that is distributed across n -data owners. Recall that this involves performing— (i) Propagate, (ii) ApplyE (iii) Gather, and (iv) ApplyV as described in §2. Observe that the state update of a vertex for this iteration depends only on the current data in its immediate neighbours and the corresponding incoming edges. Therefore, for all vertices V_i owned by a party P_i , their state update depends on the neighbours of V_i and the edges with destination in V_i . Consequently, to update the state of all vertices in V , it suffices to independently update the state of vertices in V_i for each $i \in [n]$, and then collect these updates to get the updated state of V . To facilitate this, we define subgraphs $\mathcal{G}_i = (\tilde{V}_i, E_i)$ for $i \in [n]$, where $\tilde{V}_i$ denotes the set of vertices in V_i along with their immediate neighbours and E_i denote the set of edges with destination in V_i . Note that these subgraphs, $\mathcal{G}_i = (\tilde{V}_i, E_i)$ for $i \in [n]$, are a proper subset of $\mathcal{G}$. Despite containing fewer entries than $\mathcal{G}$, assuming that the state of all the vertices in $\mathcal{G}_i$ is up to date, $\mathcal{G}_i$ contains sufficient information such that performing one iteration of message-passing allows the state of the vertices in V_i to be correctly updated. Thus, realising one iteration of message-passing computations on $\mathcal{G}$ via the DCC approach entails the following:

- (1) **Decompose:** Generating subgraphs $\mathcal{G}_i$ for $i \in [n]$ from the global graph $\mathcal{G}$.
- (2) **Compute:** Performing one iteration of message passing computations on each $\mathcal{G}_i$ for $i \in [n]$ to update the states of V_i .
- (3) **Combine:** Extracting the updated state of V_i from $\mathcal{G}_i$ for $i \in [n]$ and combining them to get the updated V .

A pictorial illustration of DCC approach for message-passing computations appears in Fig. 2.

3.2 Data Oblivious Realisation of DCC

To securely realise the DCC paradigm on a secret-shared graph, we require a data-oblivious algorithm—one whose access patterns and instruction flow depend *only* on public parameters (e.g., $|V_i|$, $|E_i|$, n), and not on the actual topology of the graph. At a high level, given an input graph G , the data-oblivious algorithm for an iteration of DCC involves (i) generating the DAG-list representation (G_i) for subgraphs $\mathcal{G}_i$, for $i \in [n]$, in the decompose step, (ii) performing message-passing computation on the DAG-lists G_i in the compute step, (iii) extracting the updated vertices V_i from G_i and updating

the V for G in the combine step. Computing the same on a secret-shared input graph G via MPC enables secure realisation of the DCC paradigm for message-passing.

Input representation: The data augmented input graph $\mathcal{G} = (V, E)$ (see §2) is represented as a set of lists $G = (V, \{E_i\}_{i \in [n]})$. Further, V is ordered as $V_1 || V_2 || \dots || V_n$, where V_i denotes all the vertices belonging to P_i and $||$ denotes concatenation. E_i for $i \in [n]$, represents the subset of edges in E that have destination in V_i (vertices belonging to P_i). Similar to prior works, each entry in V and $\{E_i\}_{i \in [n]}$ consist of a tuple (src, dst, isV, data) of four components: source (src), destination (dst), is_vertex (isV), and data. Note that this representation is itself data-oblivious and the lengths $|V_i|$, $|E_i|$ are public and independent of the graph topology.

Decompose: The decompose step takes as input the list representation of the global graph $G = (V, \{E_i\}_{i \in [n]})$ and outputs a collection of DAG-lists $\{G_i\}_{i \in [n]}$, where each G_i is a DAG-list (see 2) representing the subgraph $\mathcal{G}_i = (\tilde{V}_i, E_i)$. Here, E_i denotes all edges in G whose destination lies in V_i , and $\tilde{V}_i$ includes the vertices in V_i along with their immediate neighbours. Considering $\mathcal{G}_i$ to include only vertices in $\tilde{V}_i$ is not data-oblivious as it reveals information about the topology of G —in particular, the size $|\tilde{V}_i|$ in the view of only party P_i . Specifically, for any two subgraphs $\mathcal{G}_i$ of the same size, the $|\tilde{V}_i|$ need not be of the same size and hence is data-oblivious. To avoid this, one naive approach is to include the entire vertex set V in each subgraph. Thus, the DAG-list G_i is simply set as V concatenated with E_i . This way, for any two subgraphs $\mathcal{G}_i$ of the same size, the generated DAG-list G_i will have the same size making it data-oblivious. Algorithm 2 gives the formal details of the data-oblivious algorithm for Decompose. Note, however, that this approach increases the computation and communication as additional vertices are processed in each subgraph. Looking ahead, message-passing computations on each secret-shared subgraph G_i would incur a communication cost of $O(n|G_i|) = O(n(|V| + |E_i|))$, resulting in a total cost of $\sum_{i \in [n]} O(n(|V| + |E_i|)) = O(n^2|V| + n|E|)$, which scales quadratically with the number of parties and is thus prohibitive. We discuss in §4.2 how this can be improved.

Algorithm 2: DCC-Decompose

Input: $G = (V, \{E_i\})$ where each entry contains a tuple (src, dst, isV, data).

Output: DAG-lists $\{G_i\}_{i \in [n]}$ in vertex order.

1 **for** $i = 1$ **to** n **do in parallel**

2 $G_i = V || E_i$

Compute: This step takes as input DAG-lists of subgraph $\{G_i\}_{i \in [n]}$ and outputs updated DAG-lists after one iteration of message-passing computations. At a high level, each subgraph G_i can be processed in parallel. Further, the message-passing computations on subgraphs DAG-list G_i can be realised data-obliviously by relying on the algorithm of [11] (see Algorithm 1) with G_i as input.

Combine: This step takes as input DAG-lists G_i for $i \in [n]$ and outputs the updated graph representation $G = (V, \{E_i\}_{i \in [n]})$, where $V = V_1 || \dots || V_n$, with V_i and E_i updated as per message-passing computations in G_i . Recall that, at the end of one iteration of message-passing computations in compute, the DAG-list G_i is already sorted in vertex order i.e $G_i = V || E_i$. Since the labels of vertices in V as well

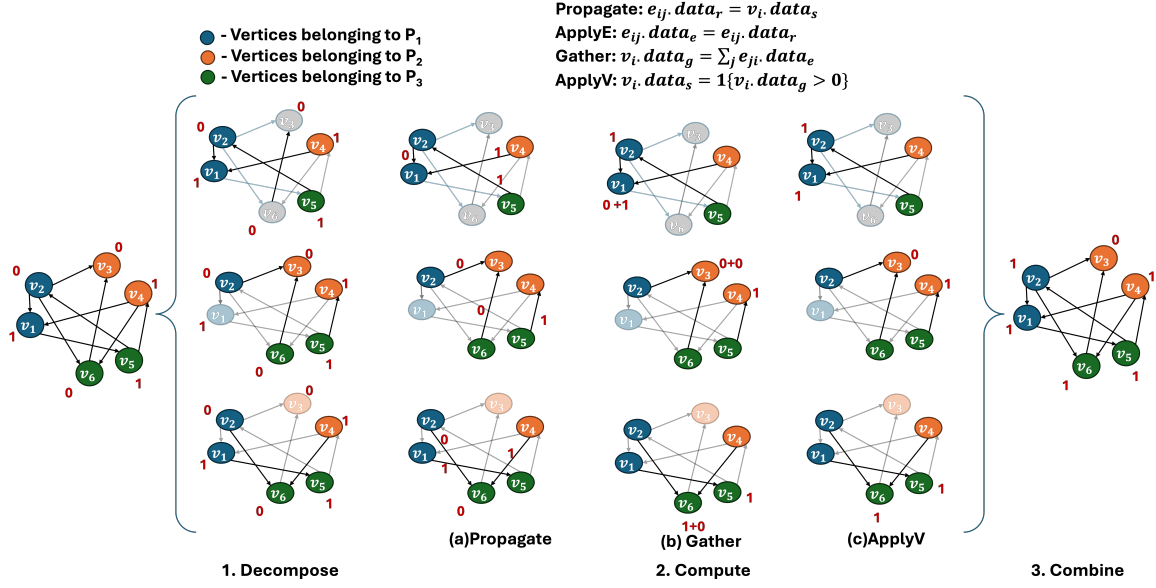


Figure 2: Example of one iteration message-passing computations using DCC.

as V_i are public information, they can be extracted from G_i . Specifically, we have, $V = G_1[1 : |V_1|] \parallel \dots \parallel G_n[\sum_{i=1}^{n-1} |V_i| + 1 : |V_n|]$ and $E_i = G_i[|V| + 1 :]$. Here, $G[l : m]$ denotes the sublist of G containing entries from $G[l]$ to $G[m]$ and $G[l :]$ denotes the sublist of G containing entries from $G[l]$ to the last entry of G . Since all accesses in this phase depend only on public values ($|V_i|$ and $|E_i|$), the combine step is naturally data-oblivious as well.

Algorithm 3: DCC-Combine

Input: DAG-lists $\{G_i = V \parallel E_i\}_{i \in [n]}$ in vertex order

Output: $G = (V, \{E_i\})$

```

1  $V = []$ 
2 for  $i = 1$  to  $n$  do
3    $ind = \sum_{j=1}^{i-1} |V_j| + 1$ 
4    $V = V \parallel G_i[ind : ind + |V_i|]$ 
5    $E_i = G_i[|V| + 1 : ]$ 
6  $G = (V, \{E_i\}_{i \in [n]})$ 

```

4 Secure Realisation of DCC paradigm

We begin by describing in §4.1 the secure realisation of the data-oblivious DCC algorithm described in §3.2. We then showcase how its communication complexity can be improved further in §4.2.

4.1 First attempt

At the start of the computation, during an input sharing phase (described in §5), the parties generate secret shares of DAG-list $G = (V, \{E_i\}_{i \in [n]})$, where each entry in these lists is additively secret-shared. Each iteration of message-passing computation begins with the secret-shared graph $\langle G \rangle = (\langle V \rangle, \{\langle E_i \rangle\}_{i \in [n]})$ and outputs updated secret shares $\langle G \rangle$, where the data components of the vertices and edges are updated. The steps involved in the secure realisation of a single iteration of our DCC approach are described next.

Decompose: This step takes as input the secret shares $\langle G \rangle = (\langle V \rangle, \{\langle E_i \rangle\}_{i \in [n]})$, and outputs the secret-shares of n DAG-lists $\langle G_i \rangle = (\langle V \rangle, \langle E_i \rangle)$, where each G_i is sorted in vertex order. Observe that since the input G is already secret shared as $(\langle V \rangle, \{\langle E_i \rangle\}_{i \in [n]})$, generating the DAG-lists G_i is non-interactive. Specifically, parties set $\langle G_i \rangle$ as $\langle G_i \rangle = \langle V \rangle \parallel \langle E_i \rangle$, where $\parallel$ denotes concatenation.

Compute: This step entails performing message-passing computations on the DAG-lists. Specifically, it takes as input the secret shares of DAG-list G_i for $i \in [n]$ and outputs its updated shares, where the data component of the vertices and edges are updated. For ease of explanation, we discuss the message-passing computations of a single DAG-list G_i . Similar computations are performed on all the DAG-lists in parallel.

Naive approach. Given $\langle G_i \rangle$, the parties can naively rely on the approach of [11] for the message-passing computations on subgraphs G_i . This involves the secure realisation of the Propagate, ApplyE, Gather, and ApplyV on $\langle G_i \rangle$. Recall that, Propagate involves non-interactive computations in vertex order of $\langle G_i \rangle$, followed by transitioning to source order of $\langle G_i \rangle$. Gather involves non-interactive computations in destination order of $\langle G_i \rangle$ followed by a transition back to vertex order of $\langle G_i \rangle$. An additional intermediate transition from source order to destination order of $\langle G_i \rangle$ is required between Propagate and Gather. Each of these transitions required for Propagate, Gather, and the intermediate step incurs a round complexity of $O(n)$ due to the reliance on secure shuffle. Further, ApplyE and ApplyV involve applying the respective functions f_{AE} and f_{AV} to each entry in the DAG-list, regardless of the order of $\langle G_i \rangle$. Specifically, let r_{AE} and r_{AV} be the round complexity required to securely apply the function f_{AE} and f_{AV} during ApplyE and ApplyV, respectively. Then ApplyE and ApplyV incur a round complexity of $O(r_{AE})$ and $O(r_{AV})$, independent of n . Thus, the overall round complexity of this approach is $O(n + r_{AE} + r_{AV})$, which scales linearly with n , constituting a bottleneck. However, we observe that the topological information of G_i , known to party P_i , can

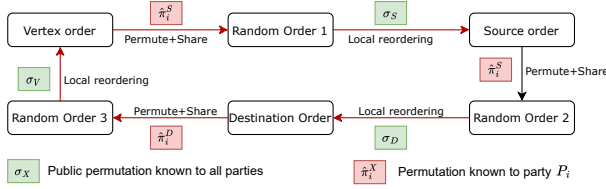


Figure 3: Transitions involved in the ‘compute’ step for G_i .

be leveraged to improve the round complexity of these transitions to be independent of n . This is described next.

Enhanced approach. We now describe how the transitions between the various orderings of G_i can be realised efficiently, with round complexity independent of n . We detail the steps involved for transitioning from vertex to source order. Similar steps can be performed for the other transitions as well. Recall that for the DAG-list G_i , party P_i is fully aware of its topology and thereby the labels of vertices and edges in the G_i . This allows P_i to define a mapping/permutation (say π_i^S) that reorders the DAG-list G_i present in vertex order to source order. This permutation is then securely applied to $\langle G_i \rangle$ to generate its source order. Since π_i^S cannot be provided in clear to other parties as it may leak topology information about G_i , the reordering is performed as follows. Parties first permute $\langle G_i \rangle$ using a random permutation $\hat{\pi}_i^S$ known to P_i . This reorders $\langle G_i \rangle$ to a random order. The random permutation is applied securely using the Permute + Share protocol, which enables permuting a secret-shared list based on a permutation known to one party and incurs a round complexity of $O(1)$. Following this, P_i computes and sends the public permutation $\sigma_i^S = \pi_i^S \circ \hat{\pi}_i^{S^{-1}}$ that maps the randomly ordered DAG-list $\langle G_i \rangle$ to source order. Since σ_i^S maps a random order of $\langle G_i \rangle$ to its source order, it does not reveal any information about the topology of G_i . The permutation σ_i^S is then applied by all parties, non-interactively, on their local shares $\langle G_i \rangle$ in the random order to generate the source order $\langle G_i \rangle$. Furthermore, note that this mapping from vertex order to source order remains consistent across multiple iterations of the message-passing phase. Hence, the same random permutation can be used in every iteration. Thus, the public permutation σ_i^S can be computed once and sent during a one-time initialisation phase. This concretely reduces the overhead for every transition by one round. A pictorial representation of all the transitions involved appears in Fig. 3. In this way, all the transitions required during one iteration of message-passing computations can be realised in $O(1)$ rounds.

Combine: This step takes as input secret shares of DAG-lists G_i for $i \in [n]$ and outputs and secret-shares of V and $\{E_i\}_{i \in [n]}$, where $V = V_1 || \dots || V_n$, with V_i and E_i updated as per message-passing computations in G_i . Recall that, at the end of one iteration of message-passing in compute, the DAG-list G_i is already sorted in vertex order. Note that labels of vertices in V as well as V_i is public information. Thus, the parties can non-interactively extract the shares of vertex entries $v \in V_i$ from shares of V in G_i . Following this, parties can non-interactively set $\langle V \rangle = \langle V_1 \rangle || \dots || \langle V_n \rangle$. Similarly, the parties can extract shares of E_i from G_i non-interactively since these are the last $|E_i|$ entries in G_i , where $|E_i|$ is public information. Thus, this step does not require any additional rounds. Finally, the secret-shares of updated V and E_i together constitute the shares of

G that is updated through one iteration of message-passing that serves as the input for the next iteration of DCC.

Formal details of the protocol described above appear in Fig. 12.

Complexity of the solution. Each iteration of message passing involves three main steps. First, ‘decompose’ where the generation of G_i is achieved non-interactively and incurs no additional communication or rounds. Second, ‘compute’ where message-passing on each G_i is performed securely by realising the operations Propagate, ApplyE, Gather, and ApplyV. For a G_i of size $O(|V| + |E_i|)$, Propagate, Gather and the intermediate transition incur $O(1)$ rounds and communication of $O(n(|V| + |E_i|))$. ApplyE incurs $O(r_{AE})$ rounds and a communication of $O((|V| + |E_i|) \cdot c_{AV})$. Similarly, ApplyV incurs $O(r_{AV})$ rounds and a communication of $O(|V| \cdot c_{AV})$. Here, r_{AE} and r_{AV} (respectively c_{AE} and c_{AV}) denote the round (respectively communication) complexity of securely computing the functions f_{AE} and f_{AV} . Thus, compute on G_i incurs $O(r_{AE} + r_{AV})$ rounds and $O(n(|V| + |E_i|) + (|V| + |E_i|) \cdot c_{AE} + |V| \cdot c_{AV})$ communication. Since computations on each G_i can be performed in parallel, the round complexity is $O(r_{AE} + r_{AV})$ rounds and total communication for this step is $O(\sum_{i=1}^n n(|V| + |E_i|) + (|V| + |E_i|) \cdot c_{AE} + |V| \cdot c_{AV}) = O(n^2|V| + n|E| + n(|V| + |E_i|) \cdot c_{AE} + n|V| \cdot c_{AV})$. Finally, during ‘combine’, the generation of updated G from G_i s is also achieved non-interactively. Therefore, the total cost incurred for one iteration of DCC is $O(r_{AE} + r_{AV})$ rounds and $O(n^2|V| + n|V| \cdot c_{AE} + n|V| \cdot c_{AV} + n|E| + |E| \cdot c_{AE})$ communication. Thus, the round complexity is independent of n , and the communication is quadratic in n .

4.2 Achieving linear communication

We now describe how to achieve a communication complexity that is linear in n . The primary bottleneck arises due to Propagate and Gather incurring quadratic communication during the ‘compute’ step. Specifically, the communication cost for Propagate and Gather in compute step for each DAG-list G_i is $O(n \cdot (|V| + |E_i|))$. Since there are n DAG-lists, the communication cost accumulates to $O(n^2|V|)$. Hence, we subsequently limit our discussion to improving communication in Propagate and Gather.

Recall from §3.1 that it suffices for each G_i to include $\bar{V}_i$ that accounts for vertices in V_i and their immediate neighbours. However, it is not data-oblivious as it leaks information about the topology of G —in particular, the size $|\bar{V}_i|$ in the view of only party P_i . To ensure privacy and efficiency, we observe that the size of $\bar{V}_i$ is bounded as $|\bar{V}_i| \leq \min(|V|, 2|E_i|)$. Therefore, we pad each $\bar{V}_i$ with additional vertex entries in V (if needed) so that its size matches this bound. This ensures that the subgraph G_i has size $O(|E_i|)$, avoiding topological leakage while also remaining communication-efficient.

However, this optimisation introduces a new challenge in the ‘decompose’ step: Recall that the ‘decompose’ step involves the generation DAG-lists G_i , which must include vertices in $\bar{V}_i$. Although the target size of $\bar{V}_i$ is now public, identifying and extracting the specific vertices that belong to $\bar{V}_i$ must also be performed in a data-oblivious manner during the ‘decompose’ step. We therefore require a data-oblivious approach that (i) collects all vertices in V_i and their neighbours in a hidden manner, and (ii) pads them to the required public size without revealing which of the extracted vertices are true neighbours.

Algorithm 4: DCC-Decompose (Improved)

Input: $G = (V, \{E_i\})$ where each entry has (src, dst, isV, data).
Output: DAG-lists $\{G_i\}_{i \in [n]}$ in vertex order.

```

1 for  $i = 1$  to  $n$  do in parallel
2    $V'_i \leftarrow \pi_i(V)$ , where  $\pi$  is the permutation that reorders  $V$ 
   such that entries in  $\bar{V}_i$  appear first.
3    $\bar{V}'_i \leftarrow V'_i[1 : \min(|V|, 2|E_i|)]$ 
4    $G_i = \bar{V}'_i || E_i$ 

```

To address this, we propose a new data-oblivious approach to realise this in Algorithm 4. Specifically, each G_i is generated from the global graph G as follows. We observe that if the vertex list is reordered such that vertices in $\bar{V}_i$ appear first, followed by the remaining vertices. Then, we extract the first $\min(|V|, 2|E_i|)$ entries from V'_i and concatenate them with E_i to form the DAG-list G_i .

In the secure realisation of Algorithm 2 using MPC, observe that the permutation π_i that has to be applied is known to party P_i , and given $\langle \cdot \rangle$ -shares of V , $\langle \cdot \rangle$ -shares of V'_i can be computed by on invocation of $\mathcal{F}_{\text{Permute}+\text{Share}}$ (similar to the transitions in compute step described in §4.1). However, since there are n DAG-lists to be generated, it requires n parallel invocations of $\mathcal{F}_{\text{Permute}+\text{Share}}$. Since each invocation of $\mathcal{F}_{\text{Permute}+\text{Share}}$ requires a communication of $O(nV)$, it incurs an overall communication of $O(n^2V)$. To improve this, we design a protocol that achieves the same goal at the cost of a single invocation of $\mathcal{F}_{\text{Permute}+\text{Share}}$. We abstract this operation as $\mathcal{F}_{\text{APS}}$. This functionality enables the generation of multiple permuted lists from a single secret-shared input list. Specifically, it takes as input a secret-shared list $\langle T \rangle$ and permutation π_i from each party P_i , and outputs n secret-shared lists $\langle T_j \rangle$ for $j = 1$ to n , such that $T_j = \pi_j(T)$. We discuss the secure protocol that realises the same in §6. Assuming $\mathcal{F}_{\text{APS}}$, we next details the secure realisation of the modified decompose step.

DAG-list generation during decompose step. Given the vertex set V , which is additively shared among all parties, the DAG-list generation proceeds as follows. Each party P_i defines a permutation π_i^G that reorders the vertex list such that the vertices in $\bar{V}_i$ (i.e., vertices associated with P_i and their immediate neighbours) appear first, followed by the remaining vertices. Note that these permutations depend on the input graph structure, which may not be available during the preprocessing phase. However, the Amortised Permute + Share protocol requires permutations to be predefined during preprocessing. To address this, the parties invoke the Amortised Permute + Share protocol, with the random permutations $\hat{\pi}_i^G$ for $i \in [n]$, which are applied to the secret-shared vertex list V in parallel. The output of this step is a set of permuted, secret-shared vertex lists, where the permutations applied are $\hat{\pi}_i^G$ for $i \in [n]$. Next, each party P_i computes and shares the public permutation $\sigma_i^G = \pi_i^G \circ \hat{\pi}_i^{G^{-1}}$ for $i \in [n]$ with all other parties. Subsequently, each party applies the public permutation σ_i^G locally to the output of the Amortised Permute + Share protocol. This final step effectively reorders the vertex list as per π_i^G , ensuring that vertices in $\bar{V}_i$ are positioned at the beginning of the list. The resulting secret-shared outputs, denoted as $\langle V'_i \rangle$ for $i = 1$ to n , represent the permuted vertex lists for each DAG-list. To generate the DAG-list for subgraph G_i , the parties extract the first $\min(|V|, 2|E_i|)$ entries

from $\langle V'_i \rangle$ and concatenate these with the corresponding secret-shared edge list $\langle E_i \rangle$. Observe here that the mapping for reordering the vertices in V remains the same across iterations. Hence, the same random permutation $\hat{\pi}_i^G$ for $i \in [n]$ can be used in every iteration. Thus, the public permutation σ_i^G for $i \in [n]$ can be computed once and sent during a one-time initialisation phase by P_i . This reduces the overhead for the subgraph generation by one round.

5 GraSP Framework

In this section, we formally describe the end-to-end protocol of GraSP for securely realising any message-passing graph algorithm. The ideal functionality for the same, $\mathcal{F}_{\text{GraSP}}$, appears in Fig. 4, and the corresponding secure protocol of GraSP, denoted as Π_{GraSP} , appears in Fig. 5. The protocol takes as input the local view of the graph from each party P_i ($i \in [n]$), consisting of its local vertices V_i and edges E_i . Additionally, all parties agree on the message-passing graph algorithm alg to be executed. Without loss of generality, we assume that all parties receive as output the updated state of their local vertices V_i ; however, the protocol can be adapted to reconstruct the output for any designated recipient. The protocol proceeds in three main phases: (1) Input Sharing and Initialisation, (2) Message-passing computations, and (3) Reconstruction, which are elaborated next.

Functionality $\mathcal{F}_{\text{GraSP}}$

Without loss of generality, let $C \subset \mathcal{P}$ denote the set of parties corrupted by adversary $\mathcal{S}$. $\mathcal{F}_{\text{GraSP}}$ interacts with parties in $\mathcal{P}$ and $\mathcal{S}$. It receives as input V_i, E_i from party P_i for $i \in [n]$. All parties in $\mathcal{P}$ agree on the graph algorithm alg , which defines the message-passing computation details such as the number of iterations (k), data to be propagated/aggregated, the functions $f_{\Delta E}, f_{\Delta V}$ etc. They send alg to $\mathcal{F}_{\text{GraSP}}$. $\mathcal{F}_{\text{GraSP}}$ proceeds as follows.

- Construct the graph $G = (V, E)$ where $V = V_1 || \dots || V_n$ and $E = E_1 || \dots || E_n$.
- For $i = 1$ to k
 - Perform k iterations of message-passing computation as defined in alg on G .
- Decompose the V as $V_1 || \dots || V_n$.
- Send $(\text{Output}, V_i, \{|V_j|, |E_j|\}_{j=1}^n)$ to the party P_i for $i = 1$ to n .

Figure 4: Ideal functionality for secure graph processing.

Input sharing: In this step, parties secret-share their input with all other parties. Specifically, $P_i \in \mathcal{P}$ generates $\langle \cdot \rangle$ -shares of V_i and E_i , as defined in §3.2. Following this, parties set $\langle G \rangle = (\langle V \rangle, \{\langle E_i \rangle\}_{i \in [n]})$ where $\langle V \rangle = \langle V_1 \rangle || \dots || \langle V_n \rangle$. Observe that, as described in §2, the generation of secret shares of V_i and E_i for $i \in [n]$ can occur non-interactively, given the shared key setup. Thus, this phase incurs no overhead in terms of rounds and communication.

Initialisation: In this phase, parties pre-compute the public permutations that are repeatedly used for DAG-list generation (during ‘decompose’) and message passing computations (during ‘compute’) of G_i , for $i \in [n]$.

- To facilitate the extraction of the vertices $\langle \bar{V}_i \rangle$ from $\langle V \rangle$ during the ‘decompose’, P_i generates mappings π_i^G and σ_i^G and distributes among the rest of the parties. Specifically, party P_i with the knowledge of the labels of vertices in $\bar{V}_i$ and their position in V , defines a mapping π_i^G which reorders the V such that all the vertices in $\bar{V}_i$

appear first, followed by other vertices. Following this, P_i samples a random permutation $\hat{\pi}_i^G$ and computes the public permutation $\sigma_i^G = \pi_i^G \circ \hat{\pi}_i^{G^{-1}}$ and sends it to all the parties³.

- Similarly, to facilitate transitions between the different orderings of $\langle G_i \rangle$ efficiently, party P_i generates and distributes among the rest of the parties certain mappings, as follows. With the knowledge of the topology of G_i , P_i defines three permutations, i.e. π_i^S - Permutation that rearranges G_i in vertex order to source order, π_i^D - Permutation that rearranges G_i in source order to destination order, π_i^V - Permutation that rearranges G_i in destination order to vertex order. Following this, P_i samples random permutations $\hat{\pi}_i^S, \hat{\pi}_i^D$ and $\hat{\pi}_i^V$ and sends the public mapping $\sigma_i^S = \pi_i^S \circ \hat{\pi}_i^{S^{-1}}, \sigma_i^D = \pi_i^D \circ \hat{\pi}_i^{D^{-1}}$ and $\sigma_i^V = \pi_i^V \circ \hat{\pi}_i^{V^{-1}}$.

Message-passing computations via DCC: Consists of multiple iterations of message-passing as required for the graph algorithm. Each iteration comprises secure realisation of ‘decompose’ (Algorithm 4), ‘compute’ (Algorithm 1) and ‘combine’ (Algorithm 3). Specifically, generation of DAG-lists $\langle G_i \rangle$ from $\langle G \rangle$ for $i \in [n]$ in ‘decompose’ step involves one call to Amortised Permute + Share as described in 4.2. The ‘compute’ step involves securely performing the message-passing computations on each subgraph G_i in parallel, as outlined in Algorithm 1. Elaborately, steps corresponding to linear operations (e.g., Steps 3,6,11, and 14) are realised non-interactively by leveraging the linearity of $\langle \cdot \rangle$ -sharing. The reordering steps invoke $\mathcal{F}_{\text{Permute+Share}}$ to apply the permutation known to party P_i ($\hat{\pi}_i^{S^{-1}}, \hat{\pi}_i^{D^{-1}}$ and $\hat{\pi}_i^{V^{-1}}$) followed by a public permutation (σ_i^S, σ_i^D , and σ_i^V respectively) provided by P_i to obtain $\langle G_i \rangle$ in the corresponding orderings. Non-linear steps are securely evaluated using $\mathcal{F}_{\text{MPC}}$ and $\mathcal{F}_{\text{mult}}$ to apply f_{AE} and update edge data, followed by $\mathcal{F}_{\text{MPC}}$ to evaluate f_{AV} on the vertex data. All updates within ApplyE and ApplyV are executed entry-wise and hence can be evaluated fully in parallel. Finally, in the ‘combine’ step, parties extract and combine the updated vertex set $\langle V_i \rangle$ and edge set $\langle E_i \rangle$ from $\langle G_i \rangle$ for $i \in [n]$ to generate $\langle G \rangle$, as described in Algorithm 3, that is used in the next iteration of DCC.

Reconstruction: After the required iterations of message-passing computations, parties reconstruct the final secret-shared state of their respective vertex sets $\langle V_i \rangle$ to obtain the updated vertex data. Depending on the application, this output can be optionally reconstructed for a subset of parties or a trusted aggregator.

Complexity of GraSP. The input sharing phase is non-interactive and does not incur any rounds or communication. The initialisation phase involves party P_i for $i \in [n]$ sending the four public permutations each of size $O(|E_i|)$ to all the parties, thus this step incurs one round and a total of $O(\sum_{i \in [n]} n|E_i|) = O(n|E|)$ communication. Each iteration of message-passing computations via DCC involves one invocation of $\mathcal{F}_{\text{APS}}$ in the ‘decompose’ step, which incurs 2 rounds and $O(n|V|)$ communication as discussed in §6. Here n denotes the number of parties. In the ‘compute’ step, message-passing computations for each subgraph G_i require 3 invocations of $\mathcal{F}_{\text{Permute+Share}}$ on a list of size $O(|E_i|)$, each of which incurs 1

round and $O(n|E_i|)$ communication. The ApplyE and ApplyV computations incur $O(r_{\text{AE}} + r_{\text{AV}})$ rounds and $O((c_{\text{AE}} + c_{\text{AV}})|E_i|)$ communication. Here, $r_{\text{AE}}, r_{\text{AV}}$ and $c_{\text{AE}}, c_{\text{AV}}$ denote the round complexity and communication cost for securely computing functions f_{AE} and f_{AV} , respectively. Thus, the ‘compute’ step incurs an overall $O(r_{\text{AE}} + r_{\text{AV}})$ rounds and $O((n + c_{\text{AE}} + c_{\text{AV}})|E|)$ communication. Finally, in during reconstruction, $O(|V_i|)$ entries are reconstructed towards P_i . This requires one round and $O(n|V|)$ communication. Hence, the total cost of securely evaluating any message-passing graph algorithm alg using GraSP is $O(k \cdot (r_{\text{AE}} + r_{\text{AV}}))$ rounds and $O(k \cdot (n + c_{\text{AE}} + c_{\text{AV}})N)$ communication, where k is the number of iterations of message-passing computations required by alg and $N = |V| + |E|$. In comparison, the state-of-the-art [11], incurs a round complexity of $O(N \log N + k \cdot (n + r_{\text{AE}} + r_{\text{AV}}))$ rounds and $O(N \log N + k \cdot (n + c_{\text{AE}} + c_{\text{AV}})N)$ communication.

GraSP achieves a round complexity that is independent of both the graph size and the number of parties, making it highly scalable. Finally, GraSP incurs communication $O(k \cdot n^2 N \log N)$ in the preprocessing phase, which is concretely better than that of [11].

Security of GraSP. We rely on the standard real-world/ideal-world simulation paradigm [13] to prove the security of our protocols. Let $\mathcal{A}$ denote the real-world adversary corrupting at most $n - 1$ parties and $\mathcal{S}$ denote the ideal-world adversary. We prove security in the *hybrid model*, where any sub-protocol Π_X is replaced by a call to its ideal functionality $\mathcal{F}_X$ [3]. Specifically, in GraSP, our protocol invokes sub-protocols Permute + Share, Amortised Permute + Share⁴, multiplication and MPC. Since secure protocols for these appear in the literature, following standard practice, we replace them with a call to their ideal functionalities, where the latter are known to be secure by definition.

Lemma 5.1: The protocol, Π_{GraSP} (Fig. 5) securely realises the functionality $\mathcal{F}_{\text{GraSP}}$ (Fig. 4) in the computational setting against a semi-honest adversary that corrupts upto $n - 1$ parties in $\mathcal{P}$, in the $(\mathcal{F}_{\text{setup}}, \mathcal{F}_{\text{MPC}}, \mathcal{F}_{\text{mult}}, \mathcal{F}_{\text{APS}}, \mathcal{F}_{\text{Permute+Share}})$ -hybrid model.

Without loss of generality, assume the adversary $\mathcal{A}$ corrupts parties $P_1, \dots, P_t$ for some $t < n$. We now construct a simulator $\mathcal{S}_{\Pi_{\text{GraSP}}}$ that generates a view for $\mathcal{A}$ that is indistinguishable from its real-world view, given only the inputs and outputs of the corrupted parties. The full description of $\mathcal{S}_{\Pi_{\text{GraSP}}}$ is given in Fig. 16. At a high level, $\mathcal{S}_{\Pi_{\text{GraSP}}}$ proceeds as follows. It emulates the setup phase using the shared randomness generated by $\mathcal{F}_{\text{setup}}$. For the initialisation phase, the simulator picks four random permutations $\sigma_i^G, \sigma_i^S, \sigma_i^D$ and σ_i^V on behalf of every honest party P_i . For DCC message-passing computations, it invokes the ideal functionalities $\mathcal{F}_{\text{APS}}, \mathcal{F}_{\text{Permute+Share}}, \mathcal{F}_{\text{MPC}}$, and $\mathcal{F}_{\text{mult}}$ on random inputs on behalf of honest parties, since these functionalities reveal no additional information about honest inputs beyond their outputs. Finally, it ensures that during the reconstruction phase, the simulated messages are consistent with the known outputs of the corrupted parties. Since all messages exchanged by honest parties in this hybrid model are either random values or outputs of ideal functionalities (which are simulated independently of honest inputs), the simulator can generate a view that is computationally indistinguishable from that of the real protocol.

³In practice, only the first $\min(|V|, 2|E_i|)$ entries of σ_i^G (that correspond to the mapping of $\hat{V}_i$) is communicated. Hence the overall cost for communicating all the permutations σ_i^G , for $i \in [n]$, is $O(n|E|)$.

⁴the security proof for Amortised Permute + Share appears in §E.

Protocol Π_{GraSP}

Inputs: Party P_i holds as input its local view of the graph V_i, E_i for $i \in [n]$. All parties in $\mathcal{P}$ agree on the graph algorithm alg , which defines the message-passing computation details such as the number of iterations for message passing (k), data to be propagated/aggregated, the functions f_{AE}, f_{AV} , etc.

Outputs: Party P_i receives V_i with data updated according to alg .

Setup: Parties invoke the $\mathcal{F}_{\text{setup}}$ for key distribution (see §A).

Input Sharing and Initialisation: Each party P_i does the following

- Generates $(\cdot)$ -shares of V_i and E_i using the shared key setup (see §A).
- Let G_i denote the subgraph in the view of party P_i with consists of edges E_i and vertices V_i that have endpoints in E_i . Each party P_i locally generates the following permutations from the topology of the subgraph G_i , (i) permutation π_i^G that reorders V such that entries in $\bar{V}_i$ appear first, (ii) permutations $\pi_i^S, \pi_i^D, \pi_i^V$ corresponding to the list representation of G_i that map vertex order $\rightarrow$ source order, source order $\rightarrow$ destination order, and destination order $\rightarrow$ vertex order.
- Samples the following random permutations: $\hat{\pi}_i^G, \hat{\pi}_i^S, \hat{\pi}_i^D, \hat{\pi}_i^V$. Computes and sends the public mapping $\sigma_i^G = \pi_i^G \circ \hat{\pi}_i^{G^{-1}}$, $\sigma_i^S = \pi_i^S \circ \hat{\pi}_i^{S^{-1}}$, $\sigma_i^D = \pi_i^D \circ \hat{\pi}_i^{D^{-1}}$, $\sigma_i^V = \pi_i^V \circ \hat{\pi}_i^{V^{-1}}$ to others.
- Generate the secret-shared list representation of the graph as $\langle G \rangle = \langle V \rangle, \{ \langle E_j \rangle \}_{j \in [n]}$ where $\langle V \rangle = \langle V_1 \rangle || \dots || \langle V_n \rangle$.

DCC message-passing computation: for k iterations:

- Decompose:**
 - Parties compute $\{ \langle V'_i \rangle \}_{i \in [n]} \leftarrow \mathcal{F}_{\text{APS}}(\langle V \rangle, \{ \hat{\pi}_i^G : P_i \}_{i \in [n]})$.
 - The parties locally apply σ_i^G on $\langle V'_i \rangle$ for $i \in [n]$.
 - Parties set $\langle \bar{V}_i \rangle = \langle V'_i \rangle [1 : \min(|V|, 2|E_i|)]$ for $i \in [n]$.
 - Parties set DAG-list $\langle G_i \rangle = \langle \bar{V}_i \rangle || \langle E_i \rangle$ for $i \in [n]$.
- Compute:** for $i = 1$ to n in parallel:
 - Propagate:** For $j = |G_i|$ to 2, parties compute $\langle G_i[j].\text{data}_r \rangle = \langle G_i[j].\text{data}_s \rangle - \langle G_i[j-1].\text{data}_s \rangle$.
 - Parties compute $\langle G_i \rangle \leftarrow \mathcal{F}_{\text{Permute+Share}}(\langle G_i \rangle, \{ P_i : \hat{\pi}_i^S \})$ followed by applying the public permutation σ_i^S on their local shares of DAG-list G_i to generate the source order.
 - For $j = |G_i|$ to 1 parties compute $\langle G_i[j].\text{data}_r \rangle = \sum_{j=1}^i \langle G_i[j].\text{data}_r \rangle - \langle G[j].\text{data}_s \rangle$.
 - ApplyE:** Parties compute $\langle G_i[j].\text{data}_e \rangle \leftarrow \mathcal{F}_{\text{MPC}}(\langle G_i[j].\text{data}_e \rangle, \langle G_i[j].\text{data}_r \rangle, f_{AE})$, $\langle G_i[j].\text{data}_e \rangle \leftarrow \mathcal{F}_{\text{mult}}(\langle G_i[j].\text{data}_e \rangle, 1 - \langle G_i[j].\text{isV} \rangle)$
 - Parties compute $\langle G_i \rangle \leftarrow \mathcal{F}_{\text{Permute+Share}}(\langle G_i \rangle, \{ P_i : \hat{\pi}_i^D \})$. They apply public permutation σ_i^D to reorder G_i by destination.
 - Gather:** Parties compute $\langle G_i[j].\text{data}_g \rangle = \sum_{j=1}^i \langle G_i[j].\text{data}_r \rangle$ for $i = |G_i|$ to 1.
 - Parties compute $\langle G_i \rangle \leftarrow \mathcal{F}_{\text{Permute+Share}}(\langle G_i \rangle, \{ P_i : \hat{\pi}_i^V \})$. They apply public permutation σ_i^V to reorder G_i by vertex order.
 - Compute $\langle G_i[j].\text{data}_g \rangle = \langle G_i[j].\text{data}_g \rangle - \langle G_i[j-1].\text{data}_g \rangle$ for $i = |V'_i|$ to 2.
 - ApplyV:** For $i = 1$ to $|V_i|$ parties compute $\langle G_i[j].\text{data}_v \rangle \leftarrow \mathcal{F}_{\text{MPC}}(\langle G_i[j].\text{data}_v \rangle, \langle G_i[j].\text{data}_g \rangle, f_{AV})$.
- Combine:**
 - For $i = 1$ to n parties compute $\langle V_i \rangle = \langle G_i \rangle [1 : |V_i|]$ and $\langle E_i \rangle = \langle G_i \rangle [l+1 : |E_i|]$, where $l = \min(V, 2|E_i|)$.
 - Parties set $\langle G \rangle = (\langle V \rangle, \{ \langle E_i \rangle \}_{i \in [n]})$ where $\langle V \rangle = \langle V_1 \rangle || \dots || \langle V_n \rangle$.

Reconstruction:

- Parties reconstruct $\langle V_i \rangle, \langle E_i \rangle$ towards P_i for $i \in [n]$. This entails every party communicating its share of V_i and E_i to party P_i .

Figure 5: GraSP protocol for secure collaborative graph processing.

6 Amortised Permute+Share

This protocol enables the generation of multiple permuted lists from a single secret-shared input list. It takes as input a secret-shared list $\langle T \rangle$ and a permutation π_i from each party P_i , and outputs n secret-shared lists $\langle T_j \rangle$ for $j = 1$ to n , such that $T_j = \pi_j(T)$.

Our protocol follows along the lines of the Permute + Share protocol of [4, 8] that allows permuting a secret-shared list using a permutation known to one party. Specifically, recall that the Permute + Share protocol takes as input a secret-shared list T and a permutation π_i known to a designated party P_i , and outputs $\langle T_O \rangle$ such that $T_O = \pi_i(T)$. The preprocessing phase involves the generation of shuffle correlation consisting of secret-shares $\langle R \rangle$ and $\langle \pi_i(R) \rangle$ for a random mask R , while the online phase involves reconstructing the masked input list $(T + R)$ towards the designated party P_i . The designated party then computes its output share as $\pi_i(T + R) - \langle \pi_i(R) \rangle_i$, while all other parties $P_j \in \mathcal{P}$ set their shares as $\langle T_O \rangle_j = -\langle \pi_i(R) \rangle_j$ for $j \neq i$.

A straightforward realisation of Amortised Permute + Share would involve running n parallel instances of Permute + Share, where each party contributes its permutation π_i in a distinct instance. This requires generating n distinct shuffle correlations $(\langle R_i \rangle, \langle \pi_i(R_i) \rangle)$ during the preprocessing phase and reconstructing n different masked lists $(T + R_i)$ in the online phase. However, since the reconstruction in Permute + Share incurs a communication of $O(nN)$, communication cost of naive approach accumulates to $O(n^2N)$. Towards improving the communication complexity, we avoid using separate masks for each party and instead use a single shared mask R not known to any party. Elaborately, during the preprocessing phase, the parties generate $\langle R \rangle$ along with $\langle \pi_1(R) \rangle, \langle \pi_2(R) \rangle, \dots, \langle \pi_n(R) \rangle$, where R is permuted according to the permutations supplied by each party. In the online phase, the parties reconstruct a single masked input, $T + R$, among all. This reconstruction of $T + R$ can be efficiently realised using the P_{king} approach [7]. Here, each party sends its share of $T + R$ to a designated king party, which reconstructs the masked list and sends it to all other parties. This approach requires only two rounds and incurs a communication cost of $O(nN)$. After reconstruction, each party P_i computes its share of the n output lists i.e $\langle T_j \rangle$ for $j = 1$ to n as follows: it sets $\langle T_j \rangle_i = \pi_i(T + R) - \langle \pi_i(R) \rangle_i$ for $j = i$ and $\langle T_j \rangle_i = -\langle \pi_i(R) \rangle_i$ for $j \neq i$. In this way, the Amortised Permute + Share protocol can be realised in $O(1)$ rounds and $O(nN)$ communication. The formal protocol for Amortised Permute + Share appears in Fig. 15 and its proof of security appears in §E.

7 Benchmarks

We implement and empirically benchmark the performance of GraSP and compare it against the state-of-the-art framework, Graphiti [11] for the application of PageRank computation. To ensure a fair comparison, both frameworks are instantiated using the generic MPC protocol of [6] in the semihonest setting. The protocols are implemented from scratch in C++ using the code base of [18]. We note that our code⁵ is specifically developed for benchmarking purposes. The benchmarks are conducted on Ubuntu servers equipped with AMD Ryzen Threadripper PRO 5965WX processors and 256GB RAM. Each party is executed as a separate process on the same

⁵https://anonymous.4open.science/r/GraSP_CCS

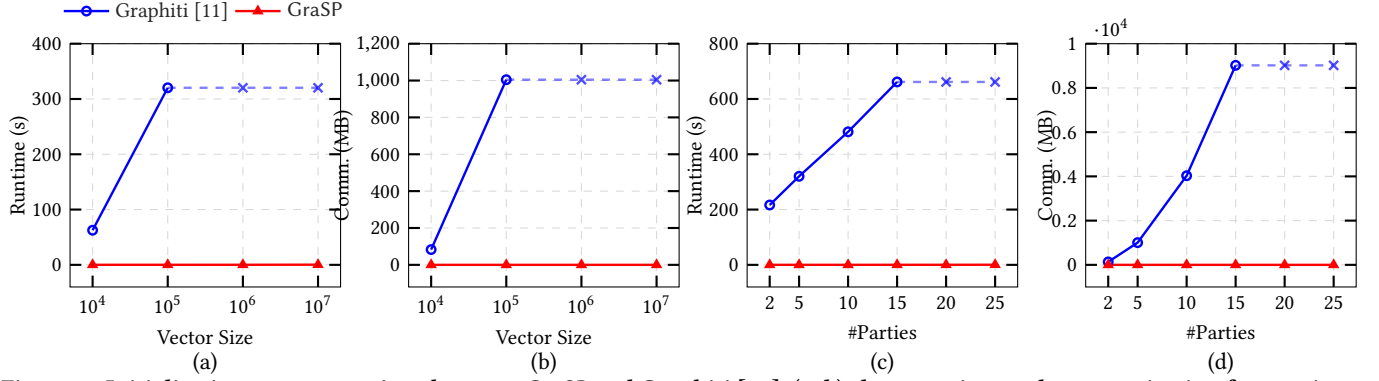


Figure 6: Initialisation cost comparison between GraSP and Graphiti [11]: (a–b) show runtime and communication for varying vector sizes (#parties = 5); (c–d) show runtime and communication for varying #parties (vector size 10^5).

machine, with the simulated network connection of 100ms latency and 100MBps bandwidth. Each party is initiated with a maximum of 10 threads. Since the performance depends only on the graph size and not its structure, we evaluate both GraSP and [11] on randomly generated scale-free graphs. Vertices are distributed equally among all parties, i.e., each party approximately owns $|V|/n$ vertices and their incident edges. We focus on the online benchmarks, while the preprocessing benchmarks are reported in §D.

Initialisations: We first compare the initialisation costs of GraSP and [11]. Since the initialisation phase of [11] has round complexity dependent on both the number of parties and the graph size, we provide comparisons by varying both parameters.

Fig. 6(a–b) showcases the comparison of both runtime and communication while varying the number of parties from 2 to 25 (with $N = |V| + |E| = 10^5$). GraSP outperforms [11] by up to 2876× in runtime and 190× in communication for 15 parties. For larger $n(>15)$, [11] runs out of memory during initialisation. The significant improvements in the runtime and communication can be attributed to the lightweight initialisation phase of GraSP, which involves only parties communicating some permutations to each other in one round of interaction. In contrast, [11] has an initialisation cost proportional to the number of parties and graph size, as it requires secure sort⁶. Fig. 13(c–d) showcases the comparison where $n = 5$ and $|V| + |E|$ varies from 10^4 to 10^7 . GraSP has up to 3200× improvement in runtime and 140× improvement in communication at 10^5 graph size. Here too, [11] runs out of memory in the initialisation phase for graph size $> 10^5$.

Message-passing: We compare the cost for one iteration of message-passing computations (Algorithm 1) using GraSP against [11]. Since runtime of [11] scales linearly with number of parties, we vary n from 2 to 25 and report the results in Fig. 7. As expected, the DCC approach in GraSP significantly outperforms [11], achieving runtime improvements of up to 14× for $n = 25$ parties. Moreover, the relative gain grows with n , from 2× for $n = 2$ to 14× for $n = 25$. This improvement stems from GraSP’s constant round complexity of 5 rounds, compared to $3n$ rounds in [11]. Although GraSP incurs a slightly higher communication than [11], its impact on runtime is negligible. This shows that the improvements in the rounds overshadow the impact of a slight increase in communication.

⁶The secure sorting operations in the initialisation phase of [11] are micro-benchmarked based on the secure variant of the quick-sort described in [1].

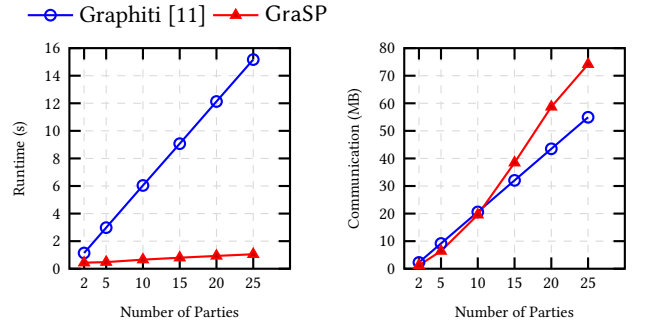


Figure 7: Comparison of one iteration of message-passing computations for varying #parties with vector size 10^5 .

End-to-end PageRank computation: For completeness, we also compare 10 iterations of PageRank computation in Fig. 13 and Tables 6 and 7. Here, we vary the number of parties, n , from 2 to 25, and the graph size ($|V| + |E|$) from 10^4 to 10^6 . GraSP outperforms [11] by up to 78× in runtime and 240× in communication for 15 parties. Overall, GraSP’s runtime remains nearly constant while varying both the number of parties and graph sizes, which can be attributed to GraSP’s constant round complexity that remains independent of these factors. However, there is a slightly higher increase in runtime for $n = 5$, and graph size 10^7 , compared to graph size of 10^6 . This discrepancy is likely due to the computational resource constraints in our experimental setup, where all parties are initialised as separate processes on the same machine. As the number of parties increases, the computational resources at each party decrease. We anticipate even better performance when each party is run on a dedicated machine. Finally, GraSP demonstrates its efficiency, scalability and practicality by processing a graph of size 10^6 distributed among 25 data owners in approximately a minute, making it highly suitable for real-world applications.

Other applications: We note that several graph algorithms such as BFS[15, 17], matrix factorization, histogram[15], contact tracing [1, 11], clustering [10], GCN [9] etc can be securely realised using GraSP. Regarding the concrete runtime for these applications, we note that the computations involved in these are very similar to those of PageRank. Specifically, steps within Propagate and Gather follow along similar lines as in the case of PageRank, while the specific functions f_{AV} and f_{AE} in ApplyE and ApplyV differ. Thus,

they are expected to show similar trends and we do not benchmark these additional applications explicitly.

8 Conclusion

We design GraSP, a generic secure framework for efficiently evaluating message-passing graph algorithms. For this, we introduce a novel Decompose-Compute-Combine (DCC) approach that leverages the knowledge of the topology of the subgraph held by a data owner (or party). To achieve the claimed efficiency and scalability, GraSP completely eliminates the need for expensive operations, such as secure sort and secure shuffle, which constituted the primary bottleneck in prior works. GraSP instead relies on a much more simpler and efficient Permute + Share primitive and further improvises it. All this enables GraSP to achieve improvements of up to 78× compared to prior works.

References

- [1] Toshinori Araki, Jun Furukawa, Kazuma Ohara, Benny Pinkas, Hanan Rosemarin, and Hikaru Tsuchida. 2021. Secure Graph Analysis at Scale. In *ACM CCS*.
- [2] Elette Boyle, Niv Gilboa, Yuval Ishai, and Ariel Nof. 2019. Practical Fully Secure Three-Party Computation via Sublinear Distributed Zero-Knowledge Proofs. In *ACM CCS*.
- [3] Ran Canetti. 2000. Security and composition of multiparty cryptographic protocols. *Journal of CRYPTOLOGY* 13, 1 (2000), 143–202.
- [4] Melissa Chase, Esha Ghosh, and Oxana Poburinnaya. 2020. Secret-shared shuffle. In *ASIACRYPT*.
- [5] Harsh Chaudhari, Ashish Choudhury, Arpita Patra, and Ajith Suresh. 2019. ASTRA: High Throughput 3PC over Rings with Application to Secure Prediction. In *ACM CCSW@CCS*.
- [6] Ivan Damgård, Marcel Keller, Enrique Larraia, Valerio Pastro, Peter Scholl, and Nigel P. Smart. 2013. Practical Covertly Secure MPC for Dishonest Majority - Or: Breaking the SPDZ Limits. In *ESORICS*. 1–18.
- [7] Ivan Damgård and Jesper Buus Nielsen. 2007. Scalable and unconditionally secure multiparty computation. In *CRYPTO*.
- [8] Saba Eskandarian and Dan Boneh. 2022. Clarion: Anonymous Communication from Multiparty Shuffling Protocols. In *NDSS*.
- [9] Nishat Koti, Varsha Bhat Kukkala, Arpita Patra, and Bhavish Raj Gopal. 2023. Entrada to Secure Graph Convolutional Networks. *Cryptology ePrint Archive* (2023).
- [10] Nishat Koti, Varsha Bhat Kukkala, Arpita Patra, Bhavish Raj Gopal, et al. 2023. Find thy neighbourhood: Privacy-preserving local clustering. *PoPETs* (2023).
- [11] Nishat Koti, Varsha Bhat Kukkala, Arpita Patra, and Bhavish Raj Gopal. 2024. Graphiti: Secure Graph Computation Made More Scalable. In *ACM CCS 2024*.
- [12] Nishat Koti, Mahak Pancholi, Arpita Patra, and Ajith Suresh. 2021. SWIFT: Super-fast and Robust Privacy-Preserving Machine Learning. In *USENIX Security*.
- [13] Yehuda Lindell. 2017. How to simulate it—a tutorial on the simulation proof technique. *Tutorials on the Foundations of Cryptography: Dedicated to Oded Goldreich* (2017), 277–346.
- [14] Payman Mohassel and Peter Rindal. 2018. ABY³: A Mixed Protocol Framework for Machine Learning. In *ACM CCS*.
- [15] Kartik Nayak, Xiao Shaun Wang, Stratis Ioannidis, Udi Weinsberg, Nina Taft, and Elaine Shi. 2015. GraphSC: Parallel secure computation made easy. In *IEEE S&P*.
- [16] Arpita Patra and Ajith Suresh. 2020. BLAZE: Blazing Fast Privacy-Preserving Machine Learning. In *NDSS*.
- [17] A Pranav Shriram, Nishat Koti, Varsha Bhat Kukkala, Arpita Patra, Bhavish Raj Gopal, and Somya Sangal. 2023. Ruffle: Rapid 3-party shuffle protocols. *PoPETs* (2023).
- [18] Xiao Wang, Alex J. Malozemoff, and Jonathan Katz. 2016. EMP-toolkit: Efficient MultiParty computation toolkit. <https://github.com/emp-toolkit>.
- [19] Zhenhua Zou, Zhuotao Liu, Jinyong Shan, Qi Li, Ke Xu, and Mingwei Xu. 2024. CoGNN: Towards Secure and Efficient Collaborative Graph Learning. In *ACM CCS 2024*.
- [20] Zhenhua Zou, Zhuotao Liu, Jinyong Shan, Qi Li, Ke Xu, and Mingwei Xu. 2025. RingSG: Optimal Secure Vertex-Centric Computation for Collaborative Graph Processing. *Cryptology ePrint Archive* (2025).

A Prelims

Shared key setup: $\mathcal{F}_{\text{setup}}$ enables the establishment of common random keys for a pseudo-random function (PRF) F among parties. This aids in non-interactively generating correlated randomness. Here $F : \{0, 1\}^k \rightarrow \mathbb{Z}_{2^{\ell}}$ is a secure PRF. The shared key setup enables all parties to non-interactively generate $\langle \cdot \rangle$ -shares of a value r . This is done by each party locally sampling a random value for its share $\langle r \rangle_i$ using the PRF key k_i and setting $r = \sum_{i \in [n]} \langle r \rangle_i$. The shared key setup also enables non-interactive generation of $\langle \cdot \rangle$ -shares of an input value x held by a party $P_i \in \mathcal{P}$. For this all parties $P_j \in \mathcal{P}$ sample a common random value jointly with P_i as their share of x using the key k_{ij} and party P_i sets its share as $\langle x \rangle_i = x - \sum_{j \in [n], j \neq i} \langle x \rangle_j$.

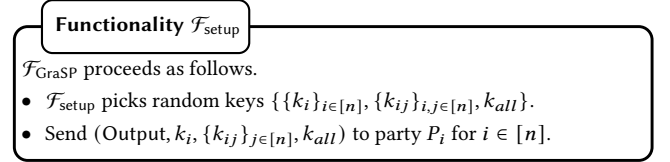


Figure 8: Ideal functionality for setup.

A.1 Notations

Symbol	Description
n	Number of parties
$\mathcal{G} = (V, E)$	Global graph
$ V , E , N$	size of vertexset, edgeset, $N = V + E $
V, E	Vertex set; edge set
V_i	Vertices owned by party P_i
E_i	Edges with destination in V_i
$\tilde{V}_i$	V_i and its neighbours (local subgraph vertices)
G	$(V, \{E_i\})$ List representation of global graph $\mathcal{G}$
DAG-list	Data augmented list representation
G_i	DAG-list representation for subgraph $\mathcal{G}_i$
$X[i : j]$	List containing entries of X from $X[i]$ to $X[j]$

Table 2: Summary of Notations

A.2 Secure computation of graph algorithms

Intuition for Propagate and Gather algorithm of [11]. At a high level, Propagate is realised by reducing “copy the source vertex’s value to all its outgoing edges” to a cumulative-sum computation. The key observation is that in source order, each vertex is immediately followed by its outgoing edges, so if one ensures that only vertex entries contribute a non-zero value (and every edge contributes 0), then a linear scan that computes the cumulative sum over all preceding entries will write a constant value across the entire block consisting of a node followed by its outgoing edges. However, a naive cumulative sum would also accumulate contributions from earlier nodes. To prevent this, the work first switches to the vertex

(1) Vertex order		$\text{data}_s'[i] \leftarrow \text{data}_s[i] - \text{data}_s[i-1], \forall i \leq V $						
	v_1	v_2	v_3	e_{12}	e_{13}	e_{23}	e_{31}	e_{32}
data_s	1	0	0	0	0	0	0	0
data_s'	1	-1	0	0	0	0	0	0
data_r	0	0	0	0	0	0	0	0

(2) Transition								
	v_1	e_{12}	e_{13}	v_2	e_{23}	v_3	e_{31}	e_{32}
data_s	1	0	0	0	0	0	0	0
data_s'	1	0	0	-1	0	0	0	0
data_r	0	1	1	0	0	0	0	0

(3) Source order		$\text{data}_r[i] \leftarrow \sum_{j \leq i} \text{data}_s'[j], \forall i \leq N $						
	v_1	e_{12}	e_{13}	v_2	e_{23}	v_3	e_{31}	e_{32}
data_s	1	0	0	0	0	0	0	0
data_s'	1	0	0	-1	0	0	0	0
data_r	0	1	1	0	0	0	0	0

Figure 9: Steps in Propagate for the example in Fig. 1.

(1) Destination order		$\text{data}_g'[i] \leftarrow \sum_{j \leq i} \text{data}_r[j], \forall i \leq N $						
	e_{31}	v_1	e_{12}	e_{32}	v_2	e_{13}	e_{23}	v_3
data_r	0	0	1	0	0	1	0	0
data_g'	0	0	1	1	1	2	2	2

(2) Transition								
	v_1	v_2	v_3	e_{12}	e_{13}	e_{23}	e_{31}	e_{32}
data_g'	0	1	2	1	2	2	0	1
data_g	0	1	1	0	0	0	0	0

(3) Vertex order		$\text{data}_g[i] \leftarrow \text{data}_g'[i] - \text{data}_g'[i-1], \forall i \leq V $						
	v_1	v_2	v_3	e_{12}	e_{13}	e_{23}	e_{31}	e_{32}
data_g'	0	1	2	1	2	2	0	1
data_g	0	1	1	0	0	0	0	0

Figure 10: Gather for the example in Fig. 1.

order, where all vertices appear contiguously (in the same relative order as they would appear in source order). In this ordering, vertex values are modified into increments rather than absolute values, i.e. the first vertex keeps its value, while every subsequent vertex stores the difference between its value and the previous vertex's value. This transformation is crucial because, when the list is reordered to source order, the cumulative sum correctly propagates the source vertex value to subsequent edges. Gather follows the dual idea for aggregation. In destination order, all incoming edges of a vertex appear contiguously before the vertex entry, so a linear scan that computes cumulative sums of the edge messages naturally accumulates the total incoming sum. But again, the cumulative sum at a vertex also contains a carry-over from everything that appeared before this node's incoming-edge block. However, the carry-over at a vertex is exactly the cumulative value already held by the previous vertex. Thus, to remove this carry-over, the list is reordered to the vertex order, where vertices are contiguous, and each vertex subtracts the previous vertex's temporary cumulative value to recover precisely the aggregation of its own incoming edges. A pictorial illustration of Propagate and Gather for message passing computation involved in identifying one-hop neighbours (see Fig. 1) appears in Fig. 9 and Fig. 10.

Initialisation phase of Graphiti. In a naive approach, transitioning between different orderings of the DAG-list would require secure sorting, which incurs significant overhead. However, [1] observed that the required transitions remain consistent across multiple message-passing rounds. Hence, the secret shares of mappings that

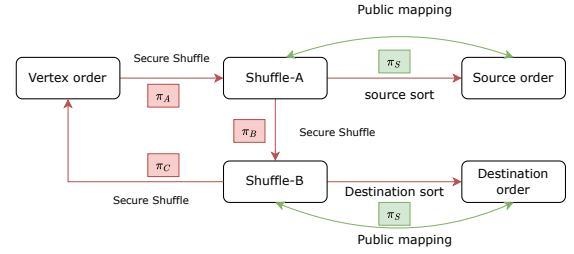


Figure 11: Initialisation in Graphiti.

allow transitioning between different orderings of the DAG-list can be generated in a one-time initialisation phase. The initialisation phase begins with the DAG-list in vertex order. Using a secure shuffle protocol, the parties apply a random permutation π^A to produce a randomised ordering, denoted as Shuffle-A. Next, another random permutation π^B is applied to Shuffle-A, resulting in a new randomised ordering, Shuffle-B. The parties then compute mappings from these randomised orderings to the source and destination orders. Specifically, the mapping from Shuffle-A to the source order is computed using a secure sort protocol. Since this mapping only links a random ordering to a sorted list, it does not reveal any sensitive information and can be made public, denoted as π^S . Similarly, the public mapping π^D from Shuffle-B to the destination order is computed. These precomputed secret-shared permutations (π^A , π^B) and public mappings (π^S , π^D) facilitate efficient transitions during the message-passing phase. Using these precomputed mappings, transitions between DAG-list orderings can be efficiently performed. For transitioning from vertex order to source order, the secret permutation π^A is applied via a secure shuffle to reorder the DAG-list into Shuffle-A, followed by locally applying the public permutation π^S to obtain the source order. Transitioning from source order to destination order involves reversing π^S via a secure shuffle to return to Shuffle-A, applying π^B to reorder into Shuffle-B, and finally applying π^D locally to reach the destination order. Transitioning from destination order to source order involves reversing π^D via a secure shuffle to return to Shuffle-B, applying π^A to reorder into Shuffle-A, and finally applying π^S locally to reach the source order. This approach ensures that transitions during message-passing are secure and efficient, with round complexity independent of the graph size. An illustration of the initialisation phase appears in Fig. 11.

Complexity of [11]: When instantiated n -party MPC protocol, initialisation phase requires $O(n + \log N)$ rounds of interaction and $O(nN \log N)$ communication. Here, n denotes the number of parties, $N = |V| + |E|$ denotes the size of G . Each iteration of message-passing computations, incurs a cost of $O(n + r_{AE} + r_{AV})$ rounds and $O(nN + c_{AE} \cdot N + c_{AV} \cdot |V|)$ communication. Here, r_x and c_x denote the round and communication complexity of computing x securely. In case of applications that require non-linear aggregation, Graphiti incurs $O(n \log |V| + r_{AE} + r_{AV})$ rounds and $O(nN \log |V| + c_{AE} \cdot N + c_{AV} \cdot |V|)$ communication. See [1, 11] for further details. Finally, in the preprocessing phase, [11] incurs communication of $(nN \log N)$ for initialisation and $O(n^2 N \log N)$ for one iteration of message-passing computations.

B First attempt

Fig. 12 gives the formal details of the secure realisation of the DCC algorithm described in §3.2.

Protocol Π_{DCC} (First attempt)

Inputs: Parties hold $\langle \cdot \rangle$ -sharing of $G = (V, \{E_i\}_{i=1}^n)$. All parties in $\mathcal{P}$ agree on the graph algorithm alg, which defines the message-passing computation details such as the number of iterations for message passing (k), data to be propagated/aggregated, the functions f_{AE} , f_{AV} etc.
Outputs: Parties hold $\langle \cdot \rangle$ -sharing of $G = (V, \{E_i\}_{i=1}^n)$ updated according to alg.

Initialisation: Each party P_i does the following

- Let G_i denote the subgraph in the view of party P_i with consists of edges E_i and vertices V_i that have endpoints in E_i . Each party P_i locally generates the permutations $\pi_i^S, \pi_i^D, \pi_i^V$ corresponding to the list representation of G_i that map vertex order $\rightarrow$ source order, source order $\rightarrow$ destination order, and destination order $\rightarrow$ vertex order.
- Samples the following random permutations: $\hat{\pi}_i^S, \hat{\pi}_i^D, \hat{\pi}_i^V$. Computes and sends the public mapping $\sigma_i^S = \pi_i^S \circ \hat{\pi}_i^{S^{-1}}, \sigma_i^D = \pi_i^D \circ \hat{\pi}_i^{D^{-1}}, \sigma_i^V = \pi_i^V \circ \hat{\pi}_i^{V^{-1}}$ to others.

DCC message-passing computation: for k iterations:

- Decompose:**
 - Parties set DAG-list $\langle G_i \rangle = \langle V \rangle \parallel \langle E_i \rangle$ for $i \in [n]$.
- Compute:** for $i = 1$ to n in parallel:
 - Propagate:** For $j = |G_i|$ to 2, parties compute $\langle G_i[j].data_r \rangle = \langle G_i[j].data_s \rangle - \langle G_i[j-1].data_s \rangle$.
 - Parties compute $\langle G_i \rangle \leftarrow \mathcal{F}_{\text{Permute+Share}}(\langle G_i \rangle, \{P_i : \hat{\pi}_i^S\})$ followed by applying the public permutation σ_i^S on their local shares of DAG-list G_i to generate the source order.
 - For $j = |G_i|$ to 1 parties compute $\langle G_i[j].data_r \rangle = \sum_{j=1}^{|G_i|} \langle G_i[j].data_r \rangle - \langle G_i[j].data_s \rangle$.
 - ApplyE:** Parties compute $\langle G_i[j].data_e \rangle \leftarrow \mathcal{F}_{\text{MPC}}(\langle G_i[j].data_e \rangle, \langle G_i[j].data_r \rangle, f_{AE})$, $\langle G_i[j].data_e \rangle \leftarrow \mathcal{F}_{\text{mult}}(\langle G_i[j].data_e \rangle, 1 - \langle G_i[j].isV \rangle)$
 - Parties compute $\langle G_i \rangle \leftarrow \mathcal{F}_{\text{Permute+Share}}(\langle G_i \rangle, \{P_i : \hat{\pi}_i^D\})$ followed by applying the public permutation σ_i^D to reorder G_i by destination.
 - Gather:** Parties compute $\langle G_i[j].data_g \rangle = \sum_{j=1}^{|G_i|} \langle G_i[j].data_r \rangle$ for $i = |G_i|$ to 1.
 - Parties compute $\langle G_i \rangle \leftarrow \mathcal{F}_{\text{Permute+Share}}(\langle G_i \rangle, \{P_i : \hat{\pi}_i^V\})$ followed by applying the public permutation σ_i^V to reorder G_i by vertex order.
 - Parties compute $\langle G_i[j].data_g \rangle = \langle G_i[j].data_g \rangle - \langle G_i[j-1].data_g \rangle$ for $i = |\tilde{V}'_i|$ to 2.
 - ApplyV:** For $i = 1$ to $|V_i|$ parties compute $\langle G_i[j].data_v \rangle \leftarrow \mathcal{F}_{\text{MPC}}(\langle G_i[j].data_v \rangle, \langle G_i[j].data_g \rangle, f_{AV})$.
- Combine:**
 - For $i = 1$ to n parties compute $\langle V \rangle = \langle G_1[1 : |V_1|] \parallel \dots \parallel \langle G_n[\sum_{i=1}^{n-1} |V_i| + 1 : |V_n|] \rangle$ and $\langle E_i \rangle = \langle G_i[|V| + 1 :] \rangle$
 - Parties set $\langle G \rangle = (\langle V \rangle, \{E_i\}_{i \in [n]})$.

Figure 12: First attempt at secure realisation of DCC for secure collaborative graph processing.

C PageRank computation via Message-passing

Here, we discuss how PageRank computation can be realised as a message passing. Given a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, the PageRank algorithm is used to rank the nodes in $\mathcal{V}$ by importance. The algorithm involves multiple iterations where the rank (state) of every node u is updated as follows: $PR(u) = \frac{1-\alpha}{|\mathcal{V}|} + \alpha \sum_{e(v,u) \in \mathcal{E}} \frac{PR(v)}{\deg(v)}$

where α is the damping factor, which is usually set to 0.85 and $\deg(v)$ is the out degree of the vertex v . The PageRank computation is expressed as a message-passing algorithm as follows. Every vertex $v \in \mathcal{V}$ ($\mathcal{V}$ with the corresponding data components) comprises two data components, one for the PageRank (PR) of the vertex and the other storing the precomputed value $\rho = \frac{1}{\deg(v)}$. Every edge $e(u, v)$ comprises a single data component $data_r$ to store the data propagated by vertices. The initial PageRank value of each vertex is set to $PR(v) = \frac{1}{|\mathcal{V}| \cdot \deg(v)}$. The algorithm proceeds iteratively, with each iteration comprising Propagate, Gather and ApplyV. An explicit call for ApplyE is not required. Propagate is defined to propagate the PR component of a vertex onto its outgoing edges i.e. $e.data_s = u.PR$ for every $e(u, v) \in \mathcal{E}$. Gather is defined to compute a cumulative sum of the $data_r$ values of all incoming edges and is stored in the PR component of the node. This ensures that each vertex receives and accumulates PageRank contributions from its neighbours. ApplyV updates the PR component of each vertex based on its aggregated value and the damping factor α . For all iterations except the last, PR component that has to be propagated is computed as:

$$v.PR = v.\rho \times \left(\frac{1-\alpha}{|\mathcal{V}|} + \alpha \cdot v.PR \right).$$

In the final iteration, ApplyV simplifies to directly updating the PageRank value as: $v.PR = \frac{1-\alpha}{|\mathcal{V}|} + \alpha \cdot v.PR$.

D Benchmarks

Table 3 and Table 4 compare the cost of initialisation for varying numbers of parties and varying graph sizes, respectively.

Ref	#Parties	Runtime (s)	Comm. (MB)
Graphiti[11]	2	212.53	133.20
GraSP		0.10	1.34
Graphiti[11]	5	320.27	1004.60
GraSP		0.10	7.17
Graphiti[11]	10	481.18	4029.54
GraSP		0.12	23.00
Graphiti[11]	15	661.60	9020.24
GraSP		0.23	46.46
Graphiti[11]	20	-	-
GraSP		0.25	73.20
Graphiti[11]	25	-	-
GraSP		0.44	97.05

Table 3: Comparison of initialisation for varying number of parties and vector size 10^5 .

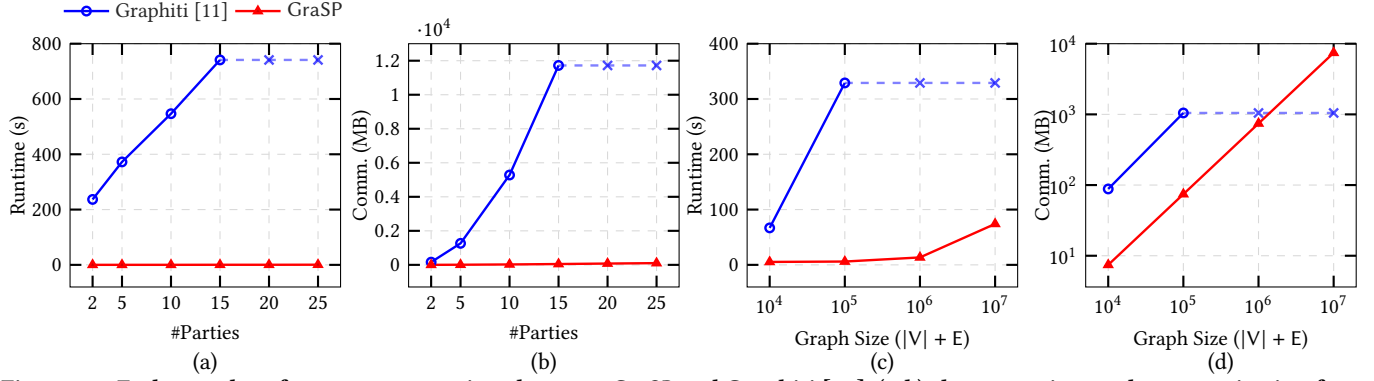


Figure 13: End-to-end performance comparison between GraSP and Graphiti [11]: (a-b) shows runtime and communication for varying #parties ($|V| + |E| = 10^5$); (c-d) shows runtime and communication for varying graph sizes (#parties=5).

$ V + E $	Ref	Runtime (s)	Comm. (MB)
10^4	Graphiti[11]	62.60	82.89
	GraSP	0.10	0.72
10^5	Graphiti[11]	320.27	1004.60
	GraSP	0.10	7.17
10^6	Graphiti[11]	-	-
	GraSP	0.12	71.72
10^7	Graphiti[11]	-	-
	GraSP	0.30	717.16

Table 4: Comparison of initialisation for varying graph size ($|V| + |E|$) and 5 parties.

Ref	#Parties	Runtime (s)	Comm. (MB)
Graphiti[11]	2	1.14	2.29
GraSP		0.44	1.26
Graphiti[11]	5	2.98	9.16
GraSP		0.48	6.41
Graphiti[11]	10	6.04	20.60
GraSP		0.66	19.57
Graphiti[11]	15	9.07	32.04
GraSP		0.80	38.45
Graphiti[11]	20	12.13	43.49
GraSP		0.93	58.71
Graphiti[11]	25	15.17	54.93
GraSP		1.05	74.16

Table 5: Comparison of one iteration of message-passing computations for varying parties and vector size 10^5 .

Table 5 compares the cost of one iteration of message-passing computations on a graph of size 10^5 while varying the number of parties from 2 to 25. Table 6 compares the end-to-end cost for 10 iterations of PageRank computation on a graph of size 10^5 while varying the number of parties from 2 to 25. Table 7 compares the end-to-end cost for 10 iterations of PageRank computation with 5 parties while varying the graph size.

Ref	#Parties	Runtime (s)	Comm. (MB)
Graphiti[11]	2	213.24	143.89
GraSP		5.54	14.69
Graphiti[11]	5	329.05	1047.33
GraSP		6.00	74.31
Graphiti[11]	10	508.97	4125.67
GraSP		7.74	225.56
Graphiti[11]	15	702.97	9169.77
GraSP		9.29	441.67
Graphiti[11]	20	-	-
GraSP		10.85	674.78
Graphiti[11]	25	-	-
GraSP		12.27	856.93

Table 6: Comparison for end-to-end performance of 10 iterations of PageRank for varying parties and graph size 10^5 .

$ V + E $	Ref	Runtime (s)	Comm. (MB)
10^4	Graphiti[11]	66.86	88.08
	GraSP	5.30	7.43
10^5	Graphiti[11]	329.05	1047.33
	GraSP	6.00	74.31
10^6	Graphiti[11]	-	-
	GraSP	13.44	743.10
10^7	Graphiti[11]	-	-
	GraSP	74.07	7431.03

Table 7: Comparison for end-to-end performance of 10 iterations of PageRank for varying graph size and 5 parties.

Preprocessing cost. For completeness we report the preprocessing cost of GraSP in Table 8. For simplicity, we instantiate the preprocessing in the presence of a trusted helper that generates the necessary correlated randomness and sends it to the computing parties in one round. In the absence of a trusted helper, the correlated randomness can be generated using generic MPC protocols in [4, 6].

Graph size	#Parties	Runtime (s)	Comm.(MB)
10^4	2	0.18	1.40
	5	0.33	1.88
	10	0.76	2.68
	15	1.38	3.48
	20	2.03	4.04
	25	2.52	4.24
10^5	2	1.21	14.00
	5	3.07	18.80
	10	7.51	26.80
	15	13.39	34.80
	20	19.62	40.40
	25	24.34	42.40
10^6	2	18.57	280.00
	5	30.73	188.00
	10	75.35	268.00
	15	135.06	348.00
	20	200.51	404.00
	25	259.05	424.00

Table 8: Preprocessing cost for 10 iterations of PageRank using GraSP for varying parties and graph size 10^5 .

E Security Proofs

We rely on the standard real-world/ideal-world simulation paradigm to prove the security of our protocols. Let $\mathcal{A}$ denote the real-world adversary corrupting at most $n - 1$ parties and $\mathcal{S}$ denote the ideal-world adversary. We prove the security of the protocol in the $\mathcal{F}_{\text{setup}}$ -hybrid model where there exists an ideal functionality $\mathcal{F}_{\text{setup}}$ to establish common PRF keys among parties in $\mathcal{P}$. This allows the parties to sample common random values among themselves non-interactively. Note that the simulation begins with the simulator $\mathcal{S}$ emulating $\mathcal{F}_{\text{setup}}$ to establish the common keys with the adversary. Since $\mathcal{S}$ has access to the inputs and randomness of $\mathcal{A}$, it can simulate the steps in the real protocol. Without loss of generality, we assume that $P_1, \dots, P_t$ for any $t < n$ are the parties corrupted by the adversary. In what follows, we first prove the security of our Amortised Permute + Share protocol, followed by the security of GraSP.

Fig. 14 presents the ideal functionality for Amortised Permute + Share and the secure protocol that realises it, Π_{APS} , appears in Fig. 15.

Functionality $\mathcal{F}_{\text{APS}}$

Without loss of generality, let $C \subset \mathcal{P}$ denote the parties corrupted by adversary $\mathcal{S}$. $\mathcal{F}_{\text{APS}}$ interacts with parties in $\mathcal{P}$ and $\mathcal{S}$. It receives as input $\langle T \rangle_i$ - P_i 's share of the input list T and π_i from party P_i for $i = 1$ to n . $\mathcal{F}_{\text{APS}}$ proceeds as follows.

- Reconstruct input T using $\langle \cdot \rangle$ -shares of parties in $\mathcal{P}$.
- Generate $T_j = \pi_j(T)$ for $j = i$ to n .
- Generate random $\langle \cdot \rangle$ -shares of T_j for $j = 1$ to n .
- Send $(\text{Output}, \langle T_j \rangle_i)$ to P_i for $i = 1$ to n and $j = 1$ to n .

Figure 14: Ideal functionality for Amortised Permute + Share.

Protocol Π_{APS}

Inputs: Party P_i inputs its $\langle \cdot \rangle$ -shares of the list T of size N and a permutation π_i .

Outputs: Additive shares of n lists $\{\langle T_j \rangle\}_{j \in [n]}$, where $T_j = \pi_j(T)$.

Preprocessing:

- Parties generate $\langle \cdot \rangle$ -shares of a random value R (see §2).
- Parties generate $\langle \cdot \rangle$ -shares of $\pi_i(R)$ for $i = 1$ to n by invoking pairwise share translation functionality $\mathcal{F}_{\text{Share Translation}}$ [4, 8].

Online:

- Parties reconstruct $T' = T + R$ using the P_{king} approach. For this, each party P_i for $i \in [n]$ locally computes $\langle T \rangle_i + \langle r \rangle_i$ and sends to the designated king party P_{king} the party P_{king} reconstructs T' and sends it back to all the parties.
- Party P_i for $i = 1$ to n sets $\langle T_i \rangle_i = \pi_i(T') - \langle \pi_i(R) \rangle_i$ and $\langle T_j \rangle_i = -\langle \pi_j(R) \rangle_i$ for $j = 1$ to n and $j \neq i$.

Figure 15: Amortised Permute + Share protocol.

Simulator $\mathcal{S}_{\Pi_{\text{GraSP}}}$

$\mathcal{S}_{\Pi_{\text{GraSP}}}$ proceeds as follows.

- **Setup:** Simulator emulates $\mathcal{F}_{\text{setup}}$ on behalf of the honest parties. Using the keys commonly held with $\mathcal{A}$ (generated as part of $\mathcal{F}_{\text{setup}}$), sample the common randomness.

- **Input Sharing and Initialisation:** On behalf of each honest party P_i , the simulator picks 4 random permutation σ_i^G of size V , σ_i^S , σ_i^D , σ_i^V of size $\min(V, 2|E_i|)$.

Message-passing computations: For k iterations do the following.

- **Decompose:** The simulator emulates $\mathcal{F}_{\text{APS}}$ with random permutations and shares as inputs on behalf of the honest parties. Note that, the inputs of the corrupt parties is available with the $\mathcal{S}_{\Pi_{\text{GraSP}}}$ as it holds the input of the corrupt parties and the common randomness generated as a part of $\mathcal{F}_{\text{setup}}$.
- **Compute:** For each subgraph G_i for $i = 1$ to n :
 - The simulator emulates $\mathcal{F}_{\text{Permute+Share}}$ for Propagate, ApplyE and Gather with random permutations and random shares as inputs on behalf of honest parties. Further, $\mathcal{S}_{\Pi_{\text{GraSP}}}$ emulates $\mathcal{F}_{\text{MPC}}$, $\mathcal{F}_{\text{mult}}$ for the computation of f_{AE} and $\mathcal{F}_{\text{MPC}}$ for computation of f_{AV} in the ApplyE and ApplyV with random shares as inputs on behalf of honest parties. Note that the other steps in Compute are non-interactive in MPC. Hence, there is nothing to simulate for these steps.
- **Combine:** There is nothing to simulate as this step is non-interactive.

Reconstruction:

- For every party $P_i \in C$, the simulator simulates the reconstruction phase by adjusting the shares of honest parties with respect to the output V_i and the corrupt parties' share. Specifically, since the $\mathcal{S}_{\Pi_{\text{GraSP}}}$ knows the input, output and the random keys generated as a part of setup phase, it can generate the shares corresponding to the output of corrupt parties. Let $\langle V_i \rangle_j$ for $j = 1$ to t denote the shares of the output. $\mathcal{S}_{\Pi_{\text{GraSP}}}$ sends random shares on behalf of honest parties $P_{t+1}, \dots, P_{n-1}$ and sends $V_i - \sum_{j=1}^{n-1} \langle V_i \rangle_j$ on behalf of P_n .

Figure 16: Simulator for GraSP protocol.

Lemma E.1: The protocol, Π_{APS} (Fig. 15) securely realises the functionality $\mathcal{F}_{\text{APS}}$ (Fig. 14) against a semi-honest adversary that corrupts at most $n - 1$ parties in $\mathcal{P}$.

PROOF. Observe that the protocol relies on invoking $\mathcal{F}_{\text{Share Translation}}$. Hence, the security follows from the security of the underlying protocols for $\mathcal{F}_{\text{Share Translation}}$. Apart from this, the only communication is in the online phase during reconstruction. Here, observe that T' and the honest parties share of T' are both lists of random values.

Thus, if P_{king} is one of the corrupt parties, then the simulator also sends random values as the shares of honest parties to P_{king} . If $P_{\text{king}} \notin C$, then the simulator picks a random list T' and sends it to all parties in the C . $\square$